

ashGPT Codebase Bundle

A multimodal LangGraph agent for Australian property law. Every Python source file, plus README, rubric, architecture doc, evaluation summary, failure analysis, and human spot check log. Generated for question bank construction (NotebookLM).

Author: Zainalabdeen Al-Saffi (48008361). Repository: ashGPT.

Table of Contents

README.md — Project overview
RUBRIC.md — Marking rubric
docs/architecture.md — Architecture overview
requirements.txt — Pinned dependencies
pyproject.toml — Pytest config
src/config.py — Centralised configuration
src/llm.py — Multi-provider LLM dispatcher
src/embeddings.py — ZeroEntropy embedding wrapper
src/indexing/build_index.py — Offline indexer
src/agent/state.py — Typed state schema
src/agent/chat_memory.py — Multi-turn chat memory
src/agent/tools.py — Retrieval tools (MMR + reranker)
src/agent/reranker.py — Cross-encoder reranker
src/agent/verification.py — Citation regex and source check
src/agent/nodes.py — Six cognitive nodes + mega-prompt ablation
src/agent/graph.py — LangGraph state machine
src/eval/run_evals.py — Evaluation harness
app.py — Streamlit frontend
tests/test_phase1_indexing.py — Phase 1 tests: indexing
tests/test_phase2_retrieval.py — Phase 2 tests: retrieval and state
tests/test_phase3_nodes.py — Phase 3 tests: nodes
tests/test_phase4_graph.py — Phase 4 tests: graph
tests/test_phase5_eval.py — Phase 5 tests: evaluation
tests/test_chat_memory.py — Chat memory tests
eval_results/eval_summary.json — Aggregate eval metrics
eval_results/failure_analysis.md — Failure analysis report
eval_results/human_spot_check.md — Human spot check log

README.md

Project overview

```
1 # ashGPT – Property Law Exam Assistant
2
3 A multimodal LangGraph agent for INFS4205 that assists law students with Australian Property Law analysis. The
  system separates legal rule extraction from chronological fact-extraction to produce grounded, structured stu
  dy aids.
4
5 ## Design Hypothesis
6
7 > *In common law property analysis, does a multi-node agent that separates chronological fact-extraction (rend
  ered via Mermaid.js) from ratio decidendi rule-extraction produce more accurate causal analyses than a single-
  prompt summarisation baseline?*
8
9 ## Architecture
10
11 The agent is a LangGraph state machine with conditional routing after retrieval. The router classifies
  intent; specialised nodes run only when needed; synthesis always grounds the final answer in retrieved source
  s.
12
13 ...
14 User Query → Router → Retrieval → ■→ Ratio Extractor ■■■→ Synthesis → END
15                               ■                               ■
16                               ■→ Chronology ■■■■■■■■
17                               ■
18                               ■→ Synthesis (general path only)
19
20 ...
21 | Intent | Path |
22 |-----|-----|
23 | `ratio` | Router → Retrieval → Ratio Extractor → Synthesis |
24 | `chronology` | Router → Retrieval → Chronology → Synthesis |
25 | `summary` | Router → Retrieval → Ratio Extractor → Chronology → Synthesis |
26 | `general` | Router → Retrieval → Synthesis |
27
28 | Node | Role |
29 |-----|-----|
30 | Router | Classifies `ratio` / `chronology` / `summary` / `general`; may suggest a `week` metadata filter
  |
31 | Retrieval | ChromaDB over text chunks and VLM-described slides; MMR + optional week/type filters |
32 | Ratio Extractor | Isolates ratio decidendi and structured IRAC from retrieved material |
33 | Chronology | Builds a timeline and emits Mermaid.js flowchart syntax |
34 | Synthesis | Produces the student-facing answer with citations; treats upstream IRAC/chronology as deriv
  ed and re-checks facts against primary sources |
35
36 ### Multi-provider LLM layer
37
38 Reasoning is not tied to a single vendor. `src/llm.py` dispatches by model name prefix (`gemini-` → Google
  GenAI, `gpt-` → OpenAI Responses API, `claude-` → Anthropic). Per-node assignments live in `src/config.py` (t
  ypical layout—edit there to experiment):
39
40 | Role | Model (as configured) |
41 |-----|-----|
42 | Slide description (indexing VLM) | `gemini-2.5-pro` |
43 | Router | `gemini-3.1-flash-lite-preview` |
44 | Ratio extractor | `gpt-5.3-chat-latest` |
45 | Chronology | `gemini-3-flash-preview` |
46 | Synthesis | `gpt-5.4-mini` |
47
48 ### Evaluation & ablation (for quantitative comparison)
49
50 `src/eval/run_evals.py` compares three configurations on the same queries:
51
52 | Configuration | What it measures |
53 |-----|-----|
54 | Full agent | Complete LangGraph pipeline |
55 | Baseline | Plain LLM (BASILINE_MODEL), no retrieval, no graph |
56 | Ablation | Retrieval + synthesis only; no router-driven ratio or chronology nodes (`intent` fixed to
  `general`) |
57
58 Query families (explicit coursework coverage): each row in `EVAL_CASES` inside `run_evals.py` is tagged wi
  th exactly one of:
59
60 | Family | What we test |
61 |-----|-----|
62 | `factual_retrieval` | Direct doctrine / tests / definitions from the indexed KB |
63 | `cross_modal_retrieval` | Questions phrased to pull lecture slide (VLM) descriptions as well as PDF text
  |
64 | `analytical_synthesis` | Compare, relate, summarise, or sequence ideas across multiple retrieved sources |
```

```

65 | `conversational_followup` | **Two-turn** scripts: agent and ablation run with `chat_history`; baseline sees
    **only the final turn** |
66
67 Outputs include `query_family`, `case_id`, and `case_rationale` per row in `eval_results.json`; `eval_summary.
    json` has `by_query_family` aggregates; `failure_analysis.md` opens with a **per-family** checklist and scores
    ; `groundedness_by_query_family.png` plots the agent by family.
68
69 **Retrieval metrics** (LLM-as-judge, binary relevance per chunk in ranked order): **precision@K**, **MRR** (re-
    ciprocal rank of the first relevant chunk), **Hit@K** (any relevant in the top-K), **NDCG@K**. Summaries and `
    retrieval_ranking_metrics.png` compare the full agent vs ablation. Optional **--retrieval-pool-eval** re-retrieves a
    larger pool (`EVAL_RETRIEVAL_POOL_K` in `config.py`), judges every chunk once, and adds **recall-vs-pool** =
    (relevant in production top-K) ÷ (relevant anywhere in pool) – a bounded proxy for recall, **not** full-corpus
    recall (document the limitation in your report).
70
71 **Automated metrics** (besides judges): heuristic **Mermaid** structural checks, **IRAC** keyword coverage, **
    latency**, **per-node latency** (agent), **token usage** (via `src/llm.py`), and on cross-modal cases **retrieval
    diagnostics** (text vs slide chunk counts).
72
73 After a run, see `eval_results/eval_summary.json` and the generated plots. Numbers change with models, prompts
    , and data–re-run the suite for your report.
74
75 ### LLM-as-a-judge framework
76
77 All judge calls go through `llm_call()` in `src/llm.py` at **temperature 0**. Each judge is instructed to return
    **only a JSON object**; the harness strips optional Markdown code fences around the payload and parses the
    result (failed parses are logged and fall back to safe defaults).
78
79 Judge models are set in `src/config.py`:
80
81 | Variable | Model (current default) | Provider |
82 |-----|-----|-----|
83 | `JUDGE_DRAFT_MODEL` | `gemini-3-flash-preview` | Google |
84 | `JUDGE_CRITIQUE_MODEL` | `gpt-5.4` | OpenAI |
85
86 **Groundedness (1-5)** – **two-stage**, cross-provider:
87
88 1. **Draft (Stage 1)** – `JUDGE_DRAFT_MODEL` receives the question, the **source material** shown to the system
    under test, and the answer. It scores how well **factual** claims (cases, dates, statutes, holdings) are supported
    by those sources; **paraphrase and plain-English explanation** are allowed. It returns `{"score": int, "reasoning":
    str}`.
89 2. **Critique (Stage 2)** – `JUDGE_CRITIQUE_MODEL` receives the same question, sources, and answer, plus the draft
    score and reasoning. It acts as a **senior reviewer**: agree with the draft or **override** the score if the draft
    was too lenient or too harsh, using the same rubric. It returns `{"score": int, "reasoning": str, "agreed_with_draft":
    bool}`.
90
91 The **final groundedness score** recorded in results is the critique model's `score` (with fallback to the draft
    score if parsing fails). Stored fields include `draft_score`, `draft_reasoning`, and `agreed_with_draft` for analysis.
92
93 **Rationale:** Using **Gemini for the first pass and OpenAI for the second** reduces **same-model self-evaluation
    bias** (e.g. a system built mainly on OpenAI being scored only by OpenAI).
94
95 **Answer relevancy (1-5)** – **single-stage**: one call using **`JUDGE_DRAFT_MODEL`** (question + answer only).
    Returns `{"score": int, "reasoning": str}`.
96
97 **Context precision @K** – **single-stage per chunk**: for each retrieved text/slide chunk, one call using **`
    JUDGE_DRAFT_MODEL`** asks whether the chunk **helps** answer the question (`{"relevant": true|false}`). Precision
    is the fraction of chunks marked relevant.
98
99 **Baseline groundedness** in the suite is judged against the **same retrieved context bundle as the full agent
    for that query (the baseline model does not see that context at generation time); this measures alignment with
    the course KB slice, not “what the baseline had access to.”
100
101 **Non-LLM structural scores** – **Mermaid validity** and **IRAC compliance** use rule-based checks over the
    dedicated node outputs or, when empty, over the **final answer** text (see `run_evals.py`).
102
103 ## Knowledge Base
104
105 The knowledge base is **strictly pre-indexed** (no runtime uploads). Two modalities:
106
107 - **Text**: PDF readings, tutorials, and notes – chunked (1500 characters, 300 overlap) and embedded with Zero
    Entropy **`zembed-1`**.
108 - **Image**: Lecture slides (**JPEG / PNG**) – described at index time by the configured **VLM**; descriptions
    are embedded as text with metadata (e.g. source, week).
109
110 Retrieval uses **maximal marginal relevance (MMR)** with tunable  $\lambda$  and fetch-**k** in `src/config.py`.
111
112 ## Setup
113
114 ### Prerequisites
115
116 - Python 3.11+
117 - Conda (recommended)

```

```

118 - API keys as required by your `src/config.py` model choices (at minimum **Google** for Gemini, **ZeroEntropy**
    * for embeddings; **OpenAI** if any `gpt-` model is used)
119
120 ### Installation
121
122 ```bash
123 conda create -n genai python=3.11 pip -y
124 conda activate genai
125 pip install -r requirements.txt
126
127 cp .env.example .env
128 # Edit .env: GOOGLE_API_KEY, ZEMBED_API_KEY, OPENAI_API_KEY, ANTHROPIC_API_KEY as needed
129 ```
130
131 ### Data layout
132
133 ```
134 data/
135 ■■■ week_1/
136 ■ ■■■ lecture/           # Slide images (.jpg / .jpeg / .png)
137 ■ ■■■ readings/         # PDFs (optional subfolder name may vary)
138 ■ ■■■ tutorial/         # PDFs
139 ■■■ week_2/
140 ■ ■■■ ...
141 ■■■ (optional root-level PDFs, e.g. supplementary notes)
142 ```
143
144 Adjust paths to match your course folder naming; the indexer expects a `week_` structure with lecture images
    and PDFs discoverable under each week.
145
146 ### Build the index
147
148 ```bash
149 python -m src.indexing.build_index
150 ```
151
152 ### Run the app
153
154 ```bash
155 streamlit run app.py
156 ```
157
158 #### Conversational memory (multi-turn)
159
160 The UI stores messages in `st.session_state`. On each send, **prior** turns are passed to `run_query(..., chat
    _history=...)` as `AgentState.chat_history`; the **current** message is always `query` alone.
161
162 - **Router** sees the transcript so follow-ups like "explain that ratio" or "draw the timeline for that case"
    resolve correctly.
163 - **Retrieval** embeds a **packed query**: the new student message plus a **short excerpt** of the last tutor
    reply (see `CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS` in `config.py`), so vector search stays on-topic without du
    mping the whole prior answer into the embedding.
164 - **Ratio, chronology, and synthesis** receive the formatted transcript so answers stay coherent across turns;
    **grounding rules** still require facts to come from retrieved sources.
165
166 Limits: `CHAT_HISTORY_MAX_MESSAGES` and `CHAT_HISTORY_MAX_CHARS_PER_MESSAGE` cap cost and context size. **Clea
    r conversation** in the sidebar wipes session memory. Memory is **per browser tab**, not persisted to disk.
167
168 The quantitative suite in `src/eval/run_evals.py` remains **single-turn** (no `chat_history`) so baseline and
    ablation comparisons stay fixed-report multi-turn behaviour as a **product / qualitative** capability unless y
    ou add dedicated multi-turn eval scenarios.
169
170 ### Evaluation
171
172 ```bash
173 python -m src.eval.run_evals --output-dir eval_results
174 ```
175
176 ### Tests
177
178 ```bash
179 pytest tests/ -v
180 pytest tests/ -v -m "not integration"      # no API keys
181 ```
182
183 ## Project Structure
184
185 ```
186 ashGPT/
187 ■■■ app.py                # Streamlit frontend
188 ■■■ src/
189 ■ ■■■ config.py          # Paths, chunking, retrieval, per-node models
190 ■ ■■■ llm.py            # Unified multi-provider LLM dispatch + token tracking

```

```

191 ■ ■■■ embeddings.py # ZeroEntropy embedding wrapper
192 ■ ■■■ indexing/
193 ■ ■■■ build_index.py # Multimodal ingestion (PDF + slide images)
194 ■ ■■■ agent/
195 ■ ■■■ state.py # LangGraph state (TypedDict)
196 ■ ■■■ chat_memory.py # Trim/format history, retrieval query packing
197 ■ ■■■ tools.py # Chroma retrieval (MMR, filters)
198 ■ ■■■ nodes.py # Router, retrieval, ratio, chronology, synthesis
199 ■ ■■■ graph.py # Conditional graph + run_query()
200 ■ ■■■ eval/
201 ■ ■■■ run_evals.py # Baseline, ablation, judges, plots, failure notes
202 ■■■ tests/
203 ■■■ data/ # Course materials (gitignored content typical)
204 ■■■ chroma_db/ # Local vector store
205 ■■■ eval_results/ # Eval JSON + PNG plots (after run_evals)
206 ■■■ requirements.txt
207 ■■■ pyproject.toml
208 ■■■ .env.example
209 ``
210
211 ## Generalisation
212
213 The hypothesis is stated in the language of common law analysis, but the underlying claim – *that separating s
tructurally distinct outputs into specialised nodes improves grounding versus a single mega-prompt* – is not s
pecific to property law. The same cognitive separation should apply to **medical case reasoning** (where a dif
ferent diagnosis ranking, a clinical timeline, and a guideline-grounded recommendation are structurally dif
ferent outputs that compete for attention inside one prompt), **financial regulatory analysis** (where a contr
ol-mapping table, a chronology of disclosure events, and an applied-rule analysis under a regulation each dema
nd a different output shape), and **scientific paper synthesis** (where a methods comparison, an evidence chro
nology, and a hypothesis-driven discussion all benefit from being generated under different system prompts and
then composed).
214
215 The pattern is simpler than the legal frame suggests: any domain in which structured outputs – chronology, tax
onomy, rule application, mapping tables, sequence diagrams – compete for attention inside a single prompt is a
candidate for cognitive separation. The IRAC + Mermaid split implemented here is one specific instance of a m
ore general design: route by the *kind of structure* the user wants, generate each structure in a node optimis
ed for it, and then synthesise. The retrieval modalities, the structural validators, and the per-node model as
signments would change between domains; the multi-node + verification + ablation evaluation pattern would not.
216
217 ## Tech Stack
218
219 | Layer | Technology |
220 |-----|-----|
221 | Orchestration | LangGraph, LangChain |
222 | Vector store | ChromaDB (local, persistent) |
223 | Embeddings | ZeroEntropy `embed-1` |
224 | LLMs / VLM | Google Gemini, OpenAI, Anthropic (per `config.py`) |
225 | Retrieval | MMR, metadata filters (week, doc type) |
226 | Frontend | Streamlit, streamlit-mermaid |
227 | Evaluation | Two-stage groundedness judge (Gemini draft → OpenAI critique), single-stage relevancy & per-chu
nk precision (Gemini), structural checks, matplotlib plots |

```

RUBRIC.md

Marking rubric

1 [FILE NOT FOUND: /sessions/vibrant-dreamy-wright/mnt/ashGPT/RUBRIC.md]

docs/architecture.md

Architecture overview

```
1 # Architecture
2
3 ## System Overview
4
5 ashGPT is a multimodal LangGraph agent that processes property law queries through a conditional state machine
6 . The architecture separates retrieval, legal rule extraction, and chronological analysis into distinct cognitive
7 nodes, enabling ablation studies and independent evaluation of each component.
8
9 Multi-turn conversation is supported via a `chat_history` field in the agent state: prior turns are trimmed,
10 validated, and packed into retrieval queries and node prompts so the agent can resolve follow-up questions without
11 conflating history with new retrieval evidence.
12
13 ## Data Flow
14
15 ```mermaid
16 graph LR
17     subgraph ingestion ["Phase 1: Offline Indexing"]
18         PDFs["PDF Readings\n& Tutorials"] -->|PyMuPDF text extraction| Chunker["Text Chunker\n1500 chars, 300 overlap"]
19         Slides["PNG Lecture\nSlides"] -->|Gemini VLM description| SlideText["Slide Descriptions"]
20         Chunker -->|zembed-1 embedding| ChromaDB["ChromaDB\nCollection"]
21         SlideText -->|zembed-1 embedding| ChromaDB
22     end
23
24
25 ## Agent Pipeline
26
27 ```mermaid
28 graph TD
29     UserQuery["User Query\n+ optional chat_history"] --> Router
30
31     subgraph pipeline ["LangGraph State Machine"]
32         Router["Router Node\nClassify intent + extract week filter"]
33         Router --> Retrieval["Retrieval Node\nMMR search: 8 text chunks + 4 slides"]
34         Retrieval --> Decision["intent?"]
35         Decision -->|ratio| RatioExtractor["Ratio Extractor\nIRAC analysis + ratio decidendi"]
36         Decision -->|chronology| Chronology["Chronology Generator\nMermaid.js flowchart"]
37         Decision -->|summary| RatioExtractor
38         Decision -->|general| Synthesis
39         RatioExtractor -->|if summary| Chronology
40         RatioExtractor -->|if ratio| Synthesis["Synthesis Node\nGrounded final answer\n(Mermaid block preserved verbatim)"]
41         Chronology --> Synthesis
42     end
43     Synthesis --> FinalAnswer["Final Answer\n+ IRAC + Mermaid diagram"]
44
45
46 ### Intent routing rules
47
48 | Intent | Trigger | Node path |
49 |-----|-----|-----|
50 | `ratio` | User asks for ratio decidendi, binding rule, or IRAC analysis | Retrieval → Ratio Extractor → Synthesis |
51 | `chronology` | User asks *only* for a diagram, timeline, or flowchart (no broader explanation) | Retrieval → Chronology → Synthesis |
52 | `summary` | User asks for an explanation/overview **or** combines a timeline request with any explanatory goal (e.g. "sequence of events and provide a summary") | Retrieval → Ratio Extractor → Chronology → Synthesis |
53 | `general` | Greetings, meta-questions, off-topic | Retrieval → Synthesis |
54
55 ## State Schema
56
57 The `AgentState` TypedDict carries all data between nodes:
58
59 ```mermaid
60 graph LR
61     subgraph inputState ["Input"]
62         Query["query: str"]
63         Week["week_filter: str | None"]
64         History["chat_history: list[ChatMessage]"]
65     end
66
67     subgraph routerState ["Router Output"]
68         Intent["intent: ratio | chronology | summary | general"]
69     end
```

```

69     end
70
71     subgraph retrievalState ["Retrieval Output"]
72         Texts["retrieved_texts: list[RetrievedDocument]"]
73         SlidesDocs["retrieved_slides: list[RetrievedDocument]"]
74     end
75
76     subgraph ratioState ["Ratio Extractor Output"]
77         Ratio["ratio_decidendi: str"]
78         IRAC["irac_analysis: str"]
79     end
80
81     subgraph chronoState ["Chronology Output"]
82         Mermaid["mermaid_diagram: str"]
83         ChronoSummary["chronology_summary: str"]
84     end
85
86     subgraph synthState ["Synthesis Output"]
87         Answer["final_answer: str"]
88     end
89
90     subgraph diagState ["Diagnostics"]
91         Trace["node_trace: list[str]"]
92     end
93
94     inputState --> routerState --> retrievalState --> ratioState --> chronoState --> synthState
95     synthState --> diagState
96 ...
97
98 `ChatMessage` is `{role: "user" | "assistant", content: str}`. History is trimmed to a maximum of 24 messages
and 3500 chars per message before being packed into the state.
99
100 ## Retrieval Strategy
101
102 ```mermaid
103 graph TD
104     QueryEmbed["Query embedded via\nzembed-1 (query mode)"] --> Candidates["Fetch top-20 candidates\nfrom Chro
maDB"]
105     Candidates --> MMR["MMR Selection\nlambda=0.5"]
106     MMR --> TextChunks["8 text chunks\nreadings, tutorials, supplementary"]
107     MMR --> SlideChunks["4 slide descriptions\nwith image_path metadata"]
108     TextChunks --> FilterMeta["Week filter?"]
109     SlideChunks --> FilterMeta
110     FilterMeta -->|"yes"| Filtered["Results restricted\nto specified week"]
111     FilterMeta -->|"no"| AllWeeks["Results from\nall weeks"]
112 ...
113
114 On follow-up turns, `build_retrieval_query` prepends an excerpt of the last assistant turn (up to 1200 chars)
to the embedding query so the vector search can resolve pronouns and contextual references without losing focu
s on the new question.
115
116 ## Multi-Turn Chat Memory
117
118 `chat_memory.py` handles three concerns independently:
119
120 - **`prepare_chat_history_for_run`** – normalises roles, strips empty messages, truncates long turns, and caps
the window at 24 messages before the history enters the state.
121 - **`format_transcript_for_llm`** – renders history as numbered `Student` / `Tutor` lines injected into router
, ratio-extractor, chronology, and synthesis prompts.
122 - **`build_retrieval_query`** – constructs a focused embedding string for the retrieval node; on follow-ups it
adds a brief prior-answer excerpt so the vector query can resolve implicit references.
123
124 The current user message always lives in `query`; prior turns live in `chat_history`. This keeps retrieval emb
eddings clean while still giving LLM nodes conversational context.
125
126 ## Streamlit Frontend
127
128 The UI (`app.py`) renders each assistant message through a `render_message()` helper that splits the stored co
ntent on ```mermaid``` fences:
129
130 - Text segments → `st.markdown`
131 - Mermaid fences → `st_mermaid` (interactive diagram)
132
133 This ensures diagrams render correctly both on first delivery and when the chat history is replayed on subsequ
ent turns. The raw Mermaid code is also available in a collapsible expander for copy-to-mermaid.live.
134
135 ## Model Assignments
136
137 | Node / Role | Model | Rationale |
138 |-----|-----|-----|
139 | VLM (slide indexing) | `gemini-2.5-pro` | Multimodal; describes slide images at index time |
140 | Router | `gemini-3.1-flash-lite-preview` | Lightweight classifier; low latency |
141 | Ratio Extractor | `gpt-5.3-chat-latest` | Mid-tier; structured IRAC output |

```

```

142 | Chronology | `gemini-3-flash-preview` | Lightweight; Mermaid generation |
143 | Synthesis | `gpt-5.4-mini` | Strong reasoning; grounded final answer |
144 | Eval baseline | `gpt-5.4-mini` | Same model as synthesis for fair comparison |
145 | Judge (draft) | `gemini-3-flash-preview` | Cross-provider stage-1 judgment |
146 | Judge (critique) | `gpt-5.4` | Stage-2 critique and score finalisation |
147
148 ## Evaluation Framework
149
150 ```mermaid
151 graph TD
152     subgraph configs ["Three Configurations"]
153         Agent["Full Agent\nAll 5 nodes"]
154         Baseline["Plain LLM Baseline\nNo retrieval, no graph"]
155         Ablation["Ablation\nRetrieval + Synthesis only"]
156     end
157
158     TestQueries["20 Test Queries\n6 weeks, 4 query families"] --> Agent
159     TestQueries --> Baseline
160     TestQueries --> Ablation
161
162     Agent --> DraftJudge["Judge Draft\ngemini-3-flash-preview\nGroundedness + Relevancy 1-5"]
163     Baseline --> DraftJudge
164     Ablation --> DraftJudge
165
166     DraftJudge --> CritiqueJudge["Judge Critique\ngpt-5.4\nFinalises score"]
167
168     CritiqueJudge --> Metrics["Metrics Export\nJSON + PNG plots"]
169     Agent -->|"per-node timing"| Metrics
170 ```
171
172 ### Query families
173
174 | Family | Description |
175 |-----|-----|
176 | `factual_retrieval` | Recall of specific cases, statutes, or holdings from the knowledge base |
177 | `cross_modal_retrieval` | Questions whose answer depends on a lecture slide diagram or figure |
178 | `analytical_synthesis` | Multi-step reasoning requiring IRAC or chronological analysis |
179 | `conversational_followup` | Follow-up turns that resolve pronouns against prior conversation |
180
181 ### Metrics collected per run
182
183 - Groundedness (LLM-as-judge, 1-5)
184 - Answer relevancy (LLM-as-judge, 1-5)
185 - Context Precision@K, MRR, Hit@K, NDCG@K (binary relevance, LLM-judged)
186 - Optional recall-vs-pool (`--retrieval-pool-eval`, expanded retrieve + judge)
187 - Source diversity (unique sources retrieved)
188 - Total latency (seconds)
189 - Per-node latency breakdown (full agent only)
190 - Node trace (which nodes fired per query)
191 - Cross-modal retrieval diagnostics on slide-targeted cases
192
193 ## Key Design Decisions
194
195 | Decision | Rationale |
196 |-----|-----|
197 | Single ChromaDB collection | Simpler retrieval with metadata filtering vs separate collections per week |
198 | MMR over similarity search | Balances relevance with source diversity; configurable for ablation |
199 | Separate per-node model assignments | Allows independent model swapping for ablation; lightweight models on cheap nodes |
200 | `summary` intent runs both reasoning nodes | A query asking for "sequence of events and a summary" needs both the Mermaid diagram and the IRAC analysis; routing to `summary` instead of `chronology` guarantees both fire |
201 | PRIMARY vs DERIVED evidence split in synthesis | Prevents the synthesis node from treating IRAC inferences as ground-truth facts |
202 | Mermaid block preserved verbatim in synthesis | Synthesis system prompt and user prompt both mandate verbatim copy of the ````mermaid` block – prevents the LLM from converting diagrams to bullet points |
203 | `render_message()` in Streamlit | All message renders (live and history replay) go through the same helpers so diagrams survive multi-turn conversations |
204 | `chat_history` separate from `query` | Keeps retrieval embeddings focused on the current question while still giving LLM nodes conversational context |
205 | Deterministic document IDs | Safe re-indexing without duplicates |
206 | `node_trace` in state | Tracks which nodes fired per query for ablation comparison and latency breakdown |
207
208 ## Beyond Property Law
209
210 The hypothesis is framed around *ratio decidendi* and chronological flowcharts, but the architectural claim it tests is domain-agnostic: **separating structurally distinct outputs into specialised nodes improves grounding versus a single mega-prompt**. This section sketches what a port to another domain would look like, using **medical case reasoning** as the worked example.
211
212 ### What stays the same
213
214 - The five-stage spine: **router → retrieval → specialised reasoning node(s) → synthesis → verification**.

```

215 - The PRIMARY-vs-DERIVED distinction in the synthesis prompt, which prevents node-generated artefacts from being treated as ground-truth facts.

216 - The ablation pattern: a **mega-prompt** baseline (one consolidated LLM call) and a **no-reranker** ablation, run alongside the full agent on the same cases, judged by a cross-provider two-stage LLM judge with a small human spot-check log for qualitative validation.

217 - The verification node: extract structured claim tokens from the final answer, look them up in the retrieved sources, and rewrite to remove or hedge unsupported claims.

218 - The metrics shape: groundedness, answer relevancy, context precision@K, MRR / NDCG, latency, source diversity, structural validity of the structured artefacts.

219

220 **### What changes when porting to medical case reasoning**

221

222 | Layer | Property Law (current) | Medical case reasoning (port) |

223 |-----|-----|-----|

224 | **Router classes** | ``ratio`` / ``chronology`` / ``summary`` / ``general`` | ``differential`` / ``timeline`` / ``guideline_application`` / ``general`` |

225 | **Specialised nodes** | Ratio Extractor (IRAC), Chronology Generator (Mermaid) | Differential Ranker (ICD-10 weighted list), Clinical Timeline (Mermaid or Gantt), Guideline Mapper (e.g. NICE / RACGP rule application) |

226 | **Retrieval modalities** | PDF readings + VLM-described lecture slides | EHR text (notes, labs, discharge summaries) + radiology images described by a VLM (chest x-ray, MRI) + structured codes (ICD-10, SNOMED) |

227 | **Structural validators** | Mermaid validity, IRAC component coverage | Differential is a ranked list with weights summing to 1; timeline events have ISO-8601 dates; guideline citation maps to a known rulebook ID |

228 | **Verification node** | Extract case citations (``X v Y``); check against retrieved chunk text | Extract clinical claims (drug name, dose, recommendation tier) and verified guideline IDs; check each against the retrieved guideline chunks |

229 | **Per-node models** | Lightweight router/chronology, mid-tier ratio, strong synthesis | Same shape, but with a clinically-tuned synthesis model (or a model with the appropriate compliance posture) |

230 | **Eval families** | factual / cross-modal / analytical / conversational | factual lookup / cross-modal (image + note) / multi-source synthesis / multi-turn handover |

231

232 **### Why the pattern carries over**

233

234 The argument generalises because the failure mode it addresses – a single prompt being asked to produce multiple structurally distinct artefacts and silently trading one for another – appears in every domain where the answer is **a composition of structured outputs grounded in a heterogeneous corpus**. Property Law happens to make that failure visible (a missing ratio is glaring; a Mermaid diagram converted to bullet points is glaring). Medicine makes it costly (a missing differential entry, a wrong dose). Financial regulation makes it auditable (a missing control mapping is a finding). The cognitive separation is the same lever in each case; only the validators and the retrieval surfaces change.

requirements.txt

Pinned dependencies

```
1 langchain>=0.3.0
2 langchain-community>=0.3.0
3 langchain-openai>=0.3.0
4 langchain-chroma>=0.2.0
5 langgraph>=0.4.0
6 chromadb>=1.0.0
7 google-generativeai>=1.70.0
8 pymupdf>=1.25.0
9 Pillow>=11.0.0
10 python-dotenv>=1.1.0
11 streamlit>=1.45.0
12 pytest>=8.0.0
13 matplotlib>=3.9.0
14 streamlit-mermaid>=0.3.0
15 anthropic>=0.40.0
16 openai>=1.50.0
17 sentence-transformers>=3.0.0
```

pyproject.toml

Pytest config

```
1 [tool.pytest.ini_options]
2 testpaths = ["tests"]
3 markers = [
4     "integration: requires live API keys",
5     "slow: takes >5 seconds",
6 ]
7 filterwarnings = [
8     "ignore::DeprecationWarning",
9     "ignore::FutureWarning",
10 ]
```

Centralised configuration

69)

src/llm.py

Multi-provider LLM dispatcher

[illegible]

[illegible]

[illegible]

src/embeddings.py

ZeroEntropy embedding wrapper

```
1  """Custom LangChain embedding wrapper for ZeroEntropy's zembed-1 model.
2
3  API reference: https://docs.zeroentropy.dev/api-reference/models/embed
4  """
5
6  from __future__ import annotations
7
8  import os
9  from typing import List
10
11  import requests
12  from langchain_core.embeddings import Embeddings
13
14  from src.config import EMBEDDING_DIMENSIONS, EMBEDDING_MODEL
15
16  ZEROENTROPY_EMBED_URL = "https://api.zeroentropy.dev/v1/models/embed"
17
18
19  class ZeroEntropyEmbeddings(Embeddings):
20      """LangChain-compatible wrapper around ZeroEntropy's embedding API.
21
22      Distinguishes between document and query input types, which allows the
23      model to optimise embeddings for asymmetric retrieval (short queries
24      matched against longer document passages).
25      """
26
27      def __init__(
28          self,
29          api_key: str | None = None,
30          model: str = EMBEDDING_MODEL,
31          dimensions: int = EMBEDDING_DIMENSIONS,
32      ) -> None:
33          self.api_key = api_key or os.getenv("ZEMBED_API_KEY")
34          if not self.api_key:
35              raise ValueError(
36                  "ZEMBED_API_KEY not found. Set it in .env or pass api_key directly."
37              )
38          self.model = model
39          self.dimensions = dimensions
40          self._headers = {
41              "Authorization": f"Bearer {self.api_key}",
42              "Content-Type": "application/json",
43          }
44
45      def _embed(self, texts: list[str], input_type: str) -> list[list[float]]:
46          """Call the ZeroEntropy embed endpoint."""
47          payload = {
48              "model": self.model,
49              "input_type": input_type,
50              "input": texts,
51              "dimensions": self.dimensions,
52          }
53          resp = requests.post(
54              ZEROENTROPY_EMBED_URL,
55              json=payload,
56              headers=self._headers,
57              timeout=120,
58          )
59          if not resp.ok:
60              raise RuntimeError(
61                  f"ZeroEntropy API error {resp.status_code}: {resp.text}"
62              )
63          results = resp.json()["results"]
64          return [r["embedding"] for r in results]
65
66      def embed_documents(self, texts: List[str]) -> List[List[float]]:
67          """Embed a list of document texts (used during indexing)."""
68          if not texts:
69              return []
70          return self._embed(texts, input_type="document")
71
72      def embed_query(self, text: str) -> List[float]:
73          """Embed a single query string (used during retrieval)."""
74          return self._embed([text], input_type="query")[0]
```

src/indexing/build_index.py

Offline indexer

[illegible]

[illegible]

```

161 """Describe and upsert all slide images from a directory. Returns count added."""
162 slides = sorted(
163     f for f in slide_dir.iterdir()
164     if f.suffix.lower() in MIME_TYPES
165 )
166 if not slides:
167     return 0
168
169 count = 0
170 for slide_path in slides:
171     try:
172         description = describe_slide(slide_path, client)
173     except Exception:
174         log.exception("VLM failed for %s – skipping", slide_path.name)
175         continue
176
177     if not description or not description.strip():
178         log.warning("■■ %s: empty VLM description – skipping", slide_path.name)
179         continue
180
181     doc_id = _sanitise_id(f"{week}_lecture_slide_{slide_path.stem}")
182     metadata = {
183         "week": week,
184         "type": "lecture_slide",
185         "source": slide_path.name,
186         "image_path": str(slide_path),
187     }
188
189     try:
190         collection.add_texts(texts=[description], ids=[doc_id], metadatas=[metadata])
191     except Exception:
192         log.exception(
193             "■■ %s: embedding failed (desc length=%d) – skipping",
194             slide_path.name,
195             len(description),
196         )
197         continue
198
199     count += 1
200     log.info("■■ %s: slide description indexed", slide_path.name)
201
202 return count
203
204
205 # ■■ Main Pipeline ████████████████████████████████████████████████████████████████████████████████
206
207
208 def build_index() -> None:
209     """Run the full multimodal ingestion pipeline."""
210     client = genai.Client() # reads GOOGLE_API_KEY from env
211
212     week_dirs = sorted(DATA_DIR.iterdir(), key=_week_sort_key)
213     week_dirs = [d for d in week_dirs if d.is_dir() and d.name.startswith("week")]
214     if not week_dirs:
215         log.error("No week_* directories found in %s", DATA_DIR)
216         sys.exit(1)
217
218     log.info("Found %d week(s) to index: %s", len(week_dirs), [d.name for d in week_dirs])
219
220     embeddings = ZeroEntropyEmbeddings()
221     persistent_client = chromadb.PersistentClient(path=str(CHROMA_DIR))
222     collection = Chroma(
223         client=persistent_client,
224         collection_name=CHROMA_COLLECTION,
225         embedding_function=embeddings,
226     )
227
228     splitter = RecursiveCharacterTextSplitter(
229         chunk_size=CHUNK_SIZE,
230         chunk_overlap=CHUNK_OVERLAP,
231         separators=["\n\n", "\n", ". ", " ", ""],
232     )
233
234     total_text_chunks = 0
235     total_slides = 0
236
237     # ■■ Index any root-level supplementary PDFs (e.g. data/*.pdf) ████████████
238     root_pdfs = sorted(DATA_DIR.glob("*.pdf"))
239     if root_pdfs:
240         log.info("■■■ Processing root-level PDFs ■■■")
241         for pdf_path in root_pdfs:
242             doc_type = _classify_pdf(pdf_path.name)
243             text = extract_text_from_pdf(pdf_path)

```

```

244 if not text.strip():
245     log.warning(" ■■ Skipping empty PDF: %s", pdf_path.name)
246     continue
247 chunks = splitter.split_text(text)
248 ids = [
249     _sanitise_id(f"root_{doc_type}_{pdf_path.stem}_chunk_{i}")
250     for i in range(len(chunks))
251 ]
252 metadatas = [
253     {
254         "week": "all",
255         "type": doc_type,
256         "source": pdf_path.name,
257         "chunk_index": i,
258     }
259     for i in range(len(chunks))
260 ]
261 collection.add_texts(texts=chunks, ids=ids, metadatas=metadatas)
262 total_text_chunks += len(chunks)
263 log.info(" ■■ %s: %d chunks indexed", pdf_path.name, len(chunks))
264
265 # ■■ Index each week
266 for week_dir in week_dirs:
267     week = week_dir.name
268     log.info("■■■ Processing %s ■■■", week)
269
270     # PDFs in subdirectories (readings/, tutorial/)
271     for subdir_name, doc_type in [("readings", "reading"), ("tutorial", "tutorial")]:
272         subdir = week_dir / subdir_name
273         if subdir.is_dir():
274             n = _ingest_pdfs(subdir, week, doc_type, splitter, collection)
275             total_text_chunks += n
276
277     # PDFs directly in the week directory (flat layout)
278     week_pdfs = sorted(week_dir.glob("*.pdf"))
279     for pdf_path in week_pdfs:
280         doc_type = _classify_pdf(pdf_path.name)
281         text = extract_text_from_pdf(pdf_path)
282         if not text.strip():
283             log.warning(" ■■ Skipping empty PDF: %s", pdf_path.name)
284             continue
285         chunks = splitter.split_text(text)
286         ids = [
287             _sanitise_id(f"{week}_{doc_type}_{pdf_path.stem}_chunk_{i}")
288             for i in range(len(chunks))
289         ]
290         metadatas = [
291             {
292                 "week": week,
293                 "type": doc_type,
294                 "source": pdf_path.name,
295                 "chunk_index": i,
296             }
297             for i in range(len(chunks))
298         ]
299         collection.add_texts(texts=chunks, ids=ids, metadatas=metadatas)
300         total_text_chunks += len(chunks)
301         log.info(" ■■ %s: %d chunks indexed", pdf_path.name, len(chunks))
302
303     # Slide images in lecture/
304     lecture_dir = week_dir / "lecture"
305     if lecture_dir.is_dir():
306         n = _ingest_slides(lecture_dir, week, collection, client)
307         total_slides += n
308
309 log.info("■■■ Indexing complete ■■■")
310 log.info(" Text chunks indexed: %d", total_text_chunks)
311 log.info(" Slide descriptions indexed: %d", total_slides)
312 log.info(" ChromaDB persisted to: %s", CHROMA_DIR.resolve())
313
314 if __name__ == "__main__":
315     build_index()

```

Typed state schema

```

1  """LangGraph state definition for the Property Law Exam Assistant.
2
3  The state flows through the graph as follows:
4
5      User Query → Router → Retrieval → [Ratio Extractor | Chronology Generator] → Synthesis
6
7  Each node reads from and writes to specific keys, keeping the data pipeline
8  explicit and inspectable for evaluation and ablation.
9  """
10
11  from __future__ import annotations
12
13  from typing import Literal, TypedDict
14
15  class RetrievedDocument(TypedDict):
16      """A single document retrieved from the knowledge base."""
17
18      content: str
19      source: str
20      week: str
21      doc_type: str
22      image_path: str | None
23
24
25  class ChatMessage(TypedDict):
26      """One turn in the session transcript (Streamlit roles map directly)."""
27
28      role: Literal["user", "assistant"]
29      content: str
30
31
32  class AgentState(TypedDict, total=False):
33      """Full state flowing through the LangGraph pipeline.
34
35      Using total=False so nodes only need to set the keys they produce.
36      Every key is optional at graph entry; the router populates intent,
37      retrieval populates documents, and downstream nodes populate their
38      respective output fields.
39      """
40
41      # ■■■ Input ■■■
42      query: str
43      week_filter: str | None
44      chat_history: list[ChatMessage] # prior turns only; current message is ``query``
45
46      # ■■■ Router output ■■■
47      intent: Literal["ratio", "chronology", "summary", "general"]
48
49      # ■■■ Retrieval output ■■■
50      retrieved_texts: list[RetrievedDocument]
51      retrieved_slides: list[RetrievedDocument]
52
53      # ■■■ Ratio Extractor output ■■■
54      ratio_decidendi: str
55      irac_analysis: str
56
57      # ■■■ Chronology Generator output ■■■
58      mermaid_diagram: str
59      chronology_summary: str
60
61      # ■■■ Synthesis output ■■■
62      final_answer: str
63
64      # ■■■ Verification node output ■■■
65      verification_report: dict
66
67      # ■■■ Ablation overrides (set by the eval harness; agent reads if present) ■
68      use_reranker: bool # if absent, retrieval_node falls back to USE_RERANKER
69
70      # ■■■ Diagnostics (for evaluation / ablation) ■■■
71      node_trace: list[str]
72

```

src/agent/chat_memory.py

Multi-turn chat memory

```
1  """Multi-turn chat helpers: trim, validate, format history, and pack retrieval queries.
2
3  ``chat_history`` in :class:`AgentState` holds **prior** turns only; the current user
4  utterance always lives in ``query``. This keeps retrieval embeddings focused while
5  still giving the router and synthesis models conversational context.
6  """
7
8  from __future__ import annotations
9
10 from src.agent.state import AgentState, ChatMessage
11 from src.config import (
12     CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS,
13     CHAT_HISTORY_MAX_CHARS_PER_MESSAGE,
14     CHAT_HISTORY_MAX_MESSAGES,
15 )
16
17
18 def prepare_chat_history_for_run(raw: list[dict[str, str]] | None) -> list[ChatMessage]:
19     """Normalise roles, strip empties, truncate long messages, cap turn count."""
20     if not raw:
21         return []
22     out: list[ChatMessage] = []
23     for m in raw:
24         role = m.get("role", "")
25         content = (m.get("content") or "").strip()
26         if role not in ("user", "assistant") or not content:
27             continue
28         cap = CHAT_HISTORY_MAX_CHARS_PER_MESSAGE
29         if len(content) > cap:
30             content = content[:cap] + "\n... [truncated]"
31         out.append({"role": role, "content": content})
32     if len(out) > CHAT_HISTORY_MAX_MESSAGES:
33         out = out[-CHAT_HISTORY_MAX_MESSAGES:]
34     return out
35
36
37 def format_transcript_for_llm(messages: list[ChatMessage]) -> str:
38     """Render history as numbered Student/Tutor lines for node prompts."""
39     if not messages:
40         return ""
41     parts: list[str] = []
42     for i, m in enumerate(messages, start=1):
43         label = "Student" if m["role"] == "user" else "Tutor"
44         parts.append(f"[{i}] {label}: {m['content']}")
45     return "\n\n".join(parts)
46
47
48 def build_retrieval_query(current_query: str, messages: list[ChatMessage]) -> str:
49     """Prefer the latest user question for embedding; add tutor excerpt on follow-ups.
50
51     Concatenating a **short** prior answer helps disambiguate "Explain that further"
52     without drowning the vector query in an entire prior diagram or IRAC block.
53     """
54     if not messages:
55         return current_query
56     for i in range(len(messages) - 1, -1, -1):
57         if messages[i]["role"] != "assistant":
58             continue
59         excerpt = messages[i]["content"][:CHAT_HISTORY_MAX_ASSISTANT_TAIL_CHARS].strip()
60         if not excerpt:
61             break
62         return (
63             "[Conversation follow-up – retrieve material relevant to both the prior "
64             "answer and the new question.]\n\n"
65             f"Prior tutor answer (excerpt):\n{excerpt}\n\n"
66             f"New student question:\n{current_query}"
67         )
68     return current_query
69
70
71 def get_chat_history(state: AgentState) -> list[ChatMessage]:
72     """Safe read of ``chat_history`` from graph state."""
73     h = state.get("chat_history")
74     return h if isinstance(h, list) else []
```

Retrieval tools (MMR + reranker)

[illegible]

[illegible]

```

161
162     rerank_on = USE_RERANKER if use_reranker is None else bool(use_reranker)
163     fetch_k = RERANKER_FETCH_K_TEXT if rerank_on else k
164
165     results = _search(store, query, k=fetch_k, where_filter=where_filter, strategy=strategy)
166     log.info(
167         "retrieve_texts [%s, rerank=%s]: %d candidates (week=%s)",
168         strategy or RETRIEVAL_STRATEGY,
169         rerank_on,
170         len(results),
171         week,
172     )
173     docs = _raw_to_retrieved(results)
174     return _maybe_rerank(query, docs, top_k=k, use_reranker=rerank_on)
175
176
177 def retrieve_slides(
178     query: str,
179     week: str | None = None,
180     k: int = 4,
181     strategy: str | None = None,
182     use_reranker: bool | None = None,
183 ) -> list[RetrievedDocument]:
184     """Retrieve lecture slide descriptions from the KB.
185
186     Returns slides whose VLM-generated descriptions are semantically similar
187     to the query. Each result carries an image_path for reference.
188
189     Args:
190         strategy: Override the global RETRIEVAL_STRATEGY for ablation.
191         use_reranker: Override the global USE_RERANKER (eval ablation hook).
192     """
193     store = _get_vectorstore()
194     where_filter = _build_filter(week=week, doc_types=["lecture_slide"])
195
196     rerank_on = USE_RERANKER if use_reranker is None else bool(use_reranker)
197     fetch_k = RERANKER_FETCH_K_SLIDES if rerank_on else k
198
199     results = _search(store, query, k=fetch_k, where_filter=where_filter, strategy=strategy)
200     log.info(
201         "retrieve_slides [%s, rerank=%s]: %d candidates (week=%s)",
202         strategy or RETRIEVAL_STRATEGY,
203         rerank_on,
204         len(results),
205         week,
206     )
207     docs = _raw_to_retrieved(results)
208     return _maybe_rerank(query, docs, top_k=k, use_reranker=rerank_on)
209
210
211 def retrieve_all(
212     query: str,
213     week: str | None = None,
214     k_text: int = 6,
215     k_slides: int = 4,
216     use_reranker: bool | None = None,
217 ) -> tuple[list[RetrievedDocument], list[RetrievedDocument]]:
218     """Convenience wrapper that retrieves both text and slides in one call."""
219     texts = retrieve_texts(query, week=week, k=k_text, use_reranker=use_reranker)
220     slides = retrieve_slides(query, week=week, k=k_slides, use_reranker=use_reranker)
221     return texts, slides

```

src/agent/reranker.py

Cross-encoder reranker

```
1  """Cross-encoder reranker for retrieved documents.
2
3  The retrieval node uses MMR for diversity. MMR optimises for marginal
4  relevance, not raw query-document similarity, so the resulting top-k can
5  include chunks that are diverse but only weakly related to the question.
6
7  This module wraps a small pretrained cross encoder (`ms-marco-MiniLM-L-6-v2`)
8  that re-scores each (query, chunk) pair and returns the top-k by score.
9  The model is loaded lazily and cached as a module-level singleton – the
10 first call pays the load cost (~80 MB download on first run); subsequent
11 calls reuse the cached instance.
12 """
13
14 from __future__ import annotations
15
16 import logging
17 from typing import Optional
18
19 from src.agent.state import RetrievedDocument
20 from src.config import RERANKER_MODEL
21
22 log = logging.getLogger(__name__)
23
24
25 _singleton: Optional["CrossEncoderReranker"] = None
26
27
28 def get_reranker() -> "CrossEncoderReranker":
29     """Return the module-level cached reranker instance (lazy init)."""
30     global _singleton
31     if _singleton is None:
32         _singleton = CrossEncoderReranker(model_name=RERANKER_MODEL)
33     return _singleton
34
35
36 class CrossEncoderReranker:
37     """Lazy cross-encoder wrapper.
38
39     The underlying ``CrossEncoder`` model is only constructed on the first
40     call to :meth:`rerank`, so importing this module is cheap.
41     """
42
43     def __init__(self, model_name: str = RERANKER_MODEL) -> None:
44         self.model_name = model_name
45         self._model = None
46
47     def _ensure_loaded(self) -> None:
48         if self._model is None:
49             try:
50                 from sentence_transformers import CrossEncoder
51             except ImportError as e:
52                 raise ImportError(
53                     "sentence-transformers is required for CrossEncoderReranker. "
54                     "Install it with `pip install sentence-transformers>=3.0.0`."
55                 ) from e
56             log.info("Loading cross-encoder reranker: %s", self.model_name)
57             self._model = CrossEncoder(self.model_name)
58
59     def rerank(
60         self,
61         query: str,
62         documents: list[RetrievedDocument],
63         top_k: int,
64     ) -> list[RetrievedDocument]:
65         """Score each document against the query, return top_k by score.
66
67         Args:
68             query: The user's question (or packed retrieval query).
69             documents: RetrievedDocument dicts from the MMR step.
70             top_k: How many documents to keep (unchanged behaviour upstream).
71
72         Returns:
73             documents sorted by cross-encoder score, truncated to top_k.
74             Empty input → empty output.
75         """
76         if not documents:
77             return []
```

```
78     if top_k <= 0:
79         return []
80
81     self._ensure_loaded()
82
83     pairs: list[tuple[str, str]] = [
84         (query, doc.get("content", "")) for doc in documents
85     ]
86
87     try:
88         scores = self._model.predict(pairs)
89     except Exception as e: # pragma: no cover – same defensive shape as llm.py
90         log.warning("Reranker failed (%s); returning original order", e)
91         return documents[:top_k]
92
93     scored = list(zip(documents, scores))
94     scored.sort(key=lambda x: float(x[1]), reverse=True)
95
96     return [doc for doc, _ in scored[:top_k]]
```

src/agent/verification.py

Citation regex and source check

```
1  """Case-citation extraction and verification helpers.
2
3  The synthesis node sometimes drifts: an LLM may import a familiar Property
4  Law case from its parametric memory even when that case is absent from the
5  retrieved sources. We catch that drift after synthesis by:
6
7  1. Extracting every plausible case citation from the final answer.
8  2. Checking each candidate against the concatenated retrieved chunk text.
9  3. Flagging citations that are not anchored in the sources so the
10     verification node can ask the synthesis model to remove or hedge them.
11
12  The regex below is intentionally permissive (italics, underscores, plain
13  text) but constrained to legal-citation shapes (`X v Y`, `X vs Y`) so
14  it does not match arbitrary "X v Y" prose.
15  """
16
17  from __future__ import annotations
18
19  import re
20
21  # A case name token: capitalised word, possibly hyphenated, possibly with
22  # articles/connectors ("of", "the") between capitalised parts. Tightened to
23  # avoid matching ordinary sentences.
24  _CASE_NAME_PART = (
25      r"[A-Z][A-Za-z'-'-]+"
26      r"(\s+(?:of|the|and|de|du|von|van)\s+[A-Z][A-Za-z'-'-]+)?"
27      r"(\s+[A-Z][A-Za-z'-'-]+){0,4}"
28  )
29
30  # Match plain "X v Y", "X vs Y", and italicised variants (*X v Y* or _X v Y_).
31  # The 'v' separator may be `v`, `v.`, `vs`, `vs.` (case-insensitive).
32  _CASE_PATTERN = re.compile(
33      r"(?P<wrap>[_*]{1,2})?"
34      rf"(?P<left>{_CASE_NAME_PART})"
35      r"\s+(?:v|vs)\s+\.?\s+"
36      rf"(?P<right>{_CASE_NAME_PART})"
37      r"(?P=wrap)?",
38      re.UNICODE,
39  )
40
41
42  # Tokens that should be trimmed if they appear at the start of a captured left
43  # side (e.g. "In Smith v Jones" → left="In Smith", trim leading "In" → "Smith").
44  # Without this, the regex's multi-word left (allowing "Buckinghamshire County
45  # Council") would absorb sentence-initial capitalised English words.
46  _LEFT_TRIM_PREFIXES = {
47      "in",
48      "the",
49      "a",
50      "an",
51      "this",
52      "that",
53      "these",
54      "those",
55      "section",
56      "see",
57      "of",
58      "cited",
59      "and",
60      "or",
61      "as",
62      "by",
63      "with",
64      "to",
65      "for",
66      "at",
67      "but",
68      "from",
69      "per",
70      "however",
71      "moreover",
72      "thus",
73  }
74
75
76  def _trim_leading_stopwords(left: str) -> str:
77      parts = left.split()
```

```

78     while parts and parts[0].lower() in _LEFT_TRIM_PREFIXES:
79         parts.pop(0)
80     return " ".join(parts)
81
82
83 def extract_case_citations(text: str) -> list[str]:
84     """Return unique, order-preserving list of case-citation strings from ``text``.
85
86     Output entries are normalised to ``<Left> v <Right>`` form (lowercased
87     separator, no italics wrappers, single spaces). Duplicates are dropped.
88     """
89     if not text:
90         return []
91
92     seen: set[str] = set()
93     out: list[str] = []
94     for m in _CASE_PATTERN.finditer(text):
95         left = re.sub(r"\s+", " ", m.group("left").strip())
96         right = re.sub(r"\s+", " ", m.group("right").strip())
97         left = _trim_leading_stopwords(left)
98         if not left or not right:
99             continue
100         canonical = f"{left} v {right}"
101         key = canonical.lower()
102         if key in seen:
103             continue
104         seen.add(key)
105         out.append(canonical)
106     return out
107
108
109 def _normalise(text: str) -> str:
110     """Lowercase + collapse whitespace + strip italic markers, for substring search."""
111     cleaned = re.sub(r"[_]", "", text or "")
112     cleaned = re.sub(r"\s+", " ", cleaned)
113     return cleaned.lower().strip()
114
115
116 def case_appears_in_sources(case: str, sources_text: str) -> bool:
117     """Return True if either the full ``X v Y`` or both party tokens occur in sources."""
118     norm_sources = _normalise(sources_text)
119     norm_case = _normalise(case)
120     if norm_case in norm_sources:
121         return True
122     parts = [p.strip() for p in re.split(r"\sv\s", norm_case) if p.strip()]
123     if len(parts) != 2:
124         return False
125     left, right = parts
126     return (left in norm_sources) and (right in norm_sources)
127
128
129 def find_unsupported_cases(answer: str, sources_text: str) -> list[str]:
130     """Return citations in ``answer`` that do not appear in ``sources_text``."""
131     candidates = extract_case_citations(answer)
132     return [c for c in candidates if not case_appears_in_sources(c, sources_text)]

```

Six cognitive nodes + mega-prompt ablation

[illegible]

```

78 # NODE 1: ROUTER
79 #
80
81
82 ROUTER_SYSTEM = (
83     "You are a query classifier for a Property Law study assistant. "
84     "You may receive a prior conversation plus a LATEST student message. "
85     "Classify only the latest message, but use the transcript to resolve "
86     "vague references (e.g. \"that case\", \"the diagram\", \"explain further\").\n\n"
87     "Return ONLY a JSON object with two keys:\n"
88     '  "intent": one of "ratio", "chronology", "summary", "general"\n'
89     '  "week_filter": a string like "week_3" if the user mentions a specific '
90     "week, or null if not.\n\n"
91     "Intent definitions:\n"
92     '- "ratio": user wants the ratio decidendi, binding legal rule, or IRAC analysis of a case.\n'
93     '- "chronology": user ONLY wants a visual timeline, flowchart, diagram, chain of title, '
94     'sequence of events, chain of events, "who did what", "what happened and when", or '
95     '"walk me through the events" – with NO request for a broader explanation or summary.\n'
96     '- "summary": use this when the user wants (a) a general case explanation OR topic overview, '
97     'OR (b) asks for BOTH a sequence/timeline AND a broader explanation or summary. '
98     'Examples that must map to "summary": "explain X and give me the sequence of events", '
99     '"who was doing what and provide a summary", "walk me through the case". '
100     'This intent runs both the chronology diagram AND the IRAC analysis, so it is the '
101     'correct choice whenever the query combines a timeline request with any explanatory goal.\n'
102     '- "general": anything else (greetings, meta-questions, off-topic).\n\n'
103     "DECISION RULE: If the query contains timeline/sequence language (e.g. \"sequence of "
104     "events\", \"who did what\", \"chain of events\", \"what happened\", \"walk me through\") "
105     "alongside any request for explanation, summary, or overview – classify as \"summary\", "
106     "NOT \"chronology\". Only use \"chronology\" when the user is asking solely for a "
107     "diagram or timeline with no broader explanation requested.\n\n"
108     "Respond with ONLY the JSON object, no markdown fences."
109 )
110
111 def router_node(state: AgentState) -> dict:
112     """Classify the user's intent and extract any week filter.
113
114     Reads: query, chat_history, week_filter (UI may pre-set week; it wins over the model)
115     Writes: intent, week_filter, node_trace
116     """
117     query = state["query"]
118     history = get_chat_history(state)
119     log.info("RouterNode: classifying query (history_turns=%d)", len(history))
120
121     if history:
122         transcript = format_transcript_for_llm(history)
123         router_input = (
124             f"CONVERSATION SO FAR:\n{transcript}\n\n"
125             f"LATEST STUDENT MESSAGE (classify this):\n{query}"
126         )
127     else:
128         router_input = query
129
130     raw = llm_call(router_input, model=ROUTER_MODEL, system_instruction=ROUTER_SYSTEM)
131
132     try:
133         cleaned = re.sub(r"```json\s*|\s*```", "", raw).strip()
134         parsed = json.loads(cleaned)
135         intent = parsed.get("intent", "general")
136         parsed_week = parsed.get("week_filter")
137     except (json.JSONDecodeError, AttributeError):
138         log.warning("Router failed to parse LLM output, defaulting to 'summary'")
139         intent = "summary"
140         parsed_week = None
141
142     valid_intents = {"ratio", "chronology", "summary", "general"}
143     if intent not in valid_intents:
144         intent = "summary"
145
146     existing_week = state.get("week_filter")
147     week_filter = existing_week if existing_week else parsed_week
148
149     log.info("RouterNode: intent=%s, week_filter=%s", intent, week_filter)
150     return {
151         "intent": intent,
152         "week_filter": week_filter,
153         "node_trace": _append_trace(state, "router"),
154     }
155
156
157 #
158 #
159 # NODE 2: RETRIEVAL
160 #

```

```

161
162
163 def retrieval_node(state: AgentState) -> dict:
164     """Query ChromaDB for relevant text chunks and slide descriptions.
165
166     Reads: query, week_filter, chat_history, use_reranker (optional override)
167     Writes: retrieved_texts, retrieved_slides, node_trace
168     """
169     history = get_chat_history(state)
170     search_q = build_retrieval_query(state["query"], history)
171     week = state.get("week_filter")
172     use_reranker_override = state.get("use_reranker")
173     log.info(
174         "RetrievalNode: searching KB (week=%s, follow_up=%s, rerank_override=%s)",
175         week,
176         bool(history),
177         use_reranker_override,
178     )
179
180     texts, slides = retrieve_all(
181         search_q,
182         week=week,
183         k_text=8,
184         k_slides=4,
185         use_reranker=use_reranker_override,
186     )
187
188     log.info(
189         "RetrievalNode: retrieved %d text chunks, %d slides",
190         len(texts),
191         len(slides),
192     )
193     return {
194         "retrieved_texts": texts,
195         "retrieved_slides": slides,
196         "node_trace": _append_trace(state, "retrieval"),
197     }
198
199
200 # =====
201 # NODE 3: RATIO EXTRACTOR
202 # =====
203
204
205 RATIO_SYSTEM = (
206     "You are an expert Australian Property Law tutor. Your task is to analyse "
207     "the provided source material and extract the ratio decidendi – the binding "
208     "legal principle that the court applied to reach its decision.\n\n"
209     "You MUST structure your response as an IRAC analysis:\n"
210     "1. Issue: State the legal question the court had to decide.\n"
211     "2. Rule: State the legal rule or principle (this IS the ratio decidendi).\n"
212     "3. Application: Explain how the court applied the rule to the facts.\n"
213     "4. Conclusion: State the court's decision.\n\n"
214     "GROUNDING RULES:\n"
215     "- FACTS must come from the sources: case names, dates, statutory references, "
216     "holdings, and specific legal tests MUST appear in the source material. "
217     "Do NOT invent cases, statutes, or factual claims.\n"
218     "- EXPLANATIONS are encouraged: you may paraphrase, simplify, and explain "
219     "legal concepts in plain English to help the student understand. This is "
220     "your role as a tutor.\n"
221     "- If you cite a case or statute by name, it MUST appear in the sources.\n"
222     "- If the source material is insufficient, say so explicitly rather than "
223     "filling gaps from your own knowledge."
224 )
225
226
227 def ratio_extractor_node(state: AgentState) -> dict:
228     """Extract the ratio decidendi and produce an IRAC analysis.
229
230     Reads: query, retrieved_texts, retrieved_slides
231     Writes: ratio_decidendi, irac_analysis, node_trace
232     """
233     query = state["query"]
234     context = _format_context(state)
235     history = get_chat_history(state)
236     hist_block = (
237         f"CONVERSATION HISTORY:\n{format_transcript_for_llm(history)}\n\n"
238         if history
239         else ""
240     )
241
242     prompt = (
243         f"{hist_block}"

```

[illegible]

```

326         else ""
327     )
328
329     prompt = (
330         f"{hist_block}"
331         f"CURRENT USER QUESTION: {query}\n\n"
332         f"SOURCE MATERIAL:\n{context}\n\n"
333         "Extract the chronological sequence of events and generate a Mermaid.js "
334         "flowchart showing the chain of title or timeline of legal events.\n\n"
335         "IMPORTANT: Only include events, dates, and parties that appear in the "
336         "source material above. Do not invent events. You may label diagram "
337         "edges with brief plain-English explanations of what happened.\n\n"
338         "Use conversation history only to interpret follow-up phrasing; facts "
339         "must come from the sources."
340     )
341
342     log.info("ChronologyNode: generating Mermaid.js diagram")
343     response = llm_call(prompt, model=CHRONOLOGY_MODEL, system_instruction=CHRONOLOGY_SYSTEM)
344
345     mermaid_match = re.search(r"```mermaid\s*\n(?:.*\n)*```", response, re.DOTALL)
346     mermaid_diagram = mermaid_match.group(1).strip() if mermaid_match else ""
347
348     summary_match = re.search(
349         r"\s*\s*Timeline Summary[:\s]*\s*(.*)",
350         response,
351         re.DOTALL | re.IGNORECASE,
352     )
353     chronology_summary = summary_match.group(1).strip() if summary_match else ""
354
355     if not mermaid_diagram:
356         log.warning("ChronologyNode: no Mermaid block found in LLM response")
357
358     return {
359         "mermaid_diagram": mermaid_diagram,
360         "chronology_summary": chronology_summary,
361         "node_trace": _append_trace(state, "chronology"),
362     }
363
364 # =====
365 # NODE 5: SYNTHESIS
366 # =====
367
368
369
370 SYNTHESIS_SYSTEM = (
371     "You are an expert Australian Property Law tutor providing a final answer "
372     "to a student's question.\n\n"
373     "You will be given:\n"
374     "- The student's original question.\n"
375     "- Retrieved source material from readings and lecture slides.\n"
376     "- Optionally, an IRAC analysis with the ratio decidendi.\n"
377     "- Optionally, a Mermaid.js chronological flowchart and timeline summary.\n\n"
378     "GROUNDING RULES:\n"
379     "- FACTS must come from the sources: all case names, dates, statutory "
380     "references, legal tests, and holdings you mention MUST appear in the "
381     "provided source material or upstream IRAC/chronology analysis. Do NOT "
382     "invent or import cases, statutes, or facts from your own knowledge.\n"
383     "- EXPLANATIONS are encouraged: paraphrase, simplify, and connect ideas "
384     "in plain English to help the student understand. You are a tutor, not "
385     "a copy machine.\n"
386     "- Use inline citations where you rely on specific sources, e.g. "
387     "(Source: Readings Week 3) or (Source: Lecture 2 Slide 5).\n"
388     "- If the sources are insufficient, say: \"The provided sources do not "
389     "cover [topic] in detail. Please consult your additional readings.\"\n\n"
390     "- If CONVERSATION HISTORY is provided, continue the same study session: resolve "
391     "pronouns and short follow-ups using the transcript, but every new factual "
392     "claim about cases or law must still appear in the PRIMARY EVIDENCE (or "
393     "verified derived analysis).\n\n"
394     "CRITICAL – MERMAID DIAGRAM PRESERVATION RULE:\n"
395     "If the DERIVED ANALYSIS section contains a ```mermaid code block, you MUST "
396     "copy that exact ```mermaid ... ``` block verbatim into your final answer. "
397     "The UI renders it as an interactive diagram – if you omit the code block or "
398     "convert it to bullet points or prose, the diagram will be lost entirely. "
399     "Do NOT paraphrase, summarise, or rewrite the Mermaid block in any form. "
400     "Place the ```mermaid block where it fits naturally in your answer "
401     "(e.g. after introducing the timeline), then continue with your explanation.\n\n"
402     "Structure your answer clearly with headings and paragraphs."
403 )
404
405
406 def synthesis_node(state: AgentState) -> dict:
407     """Compile all upstream outputs into a final answer for the student.
408

```

```

400 Reads: query, retrieved_texts, retrieved_slides, intent,
410 ratio_decidendi, irac_analysis, mermaid_diagram, chronology_summary
411 Writes: final_answer, node_trace
412 """
413 query = state["query"]
414 intent = state.get("intent", "general")
415 context = _format_context(state)
416 history = get_chat_history(state)
417
418 sections: list[str] = []
419 if history:
420     sections.append(
421         "CONVERSATION HISTORY (session memory – use to interpret follow-ups):\n"
422         f"{format_transcript_for_llm(history)}"
423     )
424     sections.append("")
425
426 sections.extend(
427     [
428         f"CURRENT STUDENT MESSAGE: {query}",
429         f"DETECTED INTENT: {intent}",
430         f"\n{' '*60}",
431         f"PRIMARY EVIDENCE (ground your answer in this):\n{context}",
432         f"\n{' '*60}",
433     ]
434 )
435
436 irac = state.get("irac_analysis", "")
437 ratio = state.get("ratio_decidendi", "")
438 mermaid = state.get("mermaid_diagram", "")
439 chrono = state.get("chronology_summary", "")
440
441 if irac or ratio or mermaid or chrono:
442     sections.append(
443         "\nDERIVED ANALYSIS (incorporate this directly into your answer – "
444         "any ``mermaid block MUST be reproduced verbatim; verify all "
445         "factual claims against the PRIMARY EVIDENCE above):"
446     )
447     if ratio:
448         sections.append(f"\nRatio Decidendi: {ratio}")
449     if irac:
450         sections.append(f"\nIRAC Analysis:\n{irac}")
451     if mermaid:
452         sections.append(f"\nChronological Flowchart:\n``mermaid\n{mermaid}\n``")
453     if chrono:
454         sections.append(f"\nTimeline Summary:\n{chrono}")
455
456 prompt = "\n".join(sections) + (
457     "\n\nSynthesise a final answer for the student. "
458     "IMPORTANT: If a ``mermaid code block appears in the DERIVED ANALYSIS, "
459     "you MUST include that exact ``mermaid ... `` block verbatim in your "
460     "response – do not convert it to bullet points or prose. "
461     "Ensure every factual claim (cases, dates, statutes, legal tests) is "
462     "supported by the PRIMARY EVIDENCE. If the derived analysis mentions "
463     "something not in the primary evidence, omit it."
464 )
465
466 log.info("SynthesisNode: compiling final answer")
467 final_answer = llm_call(prompt, model=SYNTHESIS_MODEL, system_instruction=SYNTHESIS_SYSTEM)
468
469 return {
470     "final_answer": final_answer,
471     "node_trace": _append_trace(state, "synthesis"),
472 }
473
474
475 #
476 # NODE 6: VERIFICATION (post-synthesis fact check)
477 #
478
479
480 VERIFICATION_REWRITE_SYSTEM = (
481     "You are a careful Property Law editor. You will be given:\n"
482     "1. The student's question.\n"
483     "2. The retrieved source material.\n"
484     "3. A draft answer.\n"
485     "4. A list of CASE CITATIONS in the draft that DO NOT appear in the sources.\n\n"
486     "Your task: rewrite the draft answer so that EACH unsupported citation is "
487     "either (a) removed entirely, or (b) hedged with a phrase that signals it is "
488     "not in the provided materials (e.g. \"is not covered in the indexed "
489     "sources\"). Preserve every supported claim, the structure of the answer, "
490     "every Markdown heading, and any ``mermaid code block VERBATIM. Do not "
491     "introduce new citations. Do not mention this rewrite process to the "

```

```

492     "student. Return ONLY the rewritten answer."
493 )
494
495
496 def _collect_sources_text(state: AgentState) -> str:
497     """Concatenate raw content from retrieved texts and slides for citation lookup."""
498     parts: list[str] = []
499     for doc in state.get("retrieved_texts", []) or []:
500         parts.append(doc.get("content", ""))
501         parts.append(doc.get("source", ""))
502     for doc in state.get("retrieved_slides", []) or []:
503         parts.append(doc.get("content", ""))
504         parts.append(doc.get("source", ""))
505     return "\n".join(p for p in parts if p)
506
507
508 def verification_node(state: AgentState) -> dict:
509     """Fact-check cited cases in the final answer against retrieved sources.
510
511     For each italicised or plain ``X v Y`` citation in ``final_answer``, check
512     whether the case name appears anywhere in the retrieved chunk content.
513     If unsupported citations exist, ask the synthesis model to rewrite the
514     answer with those claims removed or hedged. Supported claims are
515     preserved verbatim by prompt design.
516
517     Reads: final_answer, retrieved_texts, retrieved_slides, query
518     Writes: final_answer (possibly rewritten), verification_report, node_trace
519     """
520     answer = state.get("final_answer", "") or ""
521     sources_text = _collect_sources_text(state)
522     unsupported = find_unsupported_cases(answer, sources_text)
523
524     report = {
525         "unsupported_claims": unsupported,
526         "rewrites_applied": False,
527     }
528
529     if not unsupported:
530         log.info("VerificationNode: all citations supported – no rewrite needed")
531         return {
532             "verification_report": report,
533             "node_trace": _append_trace(state, "verification"),
534         }
535
536     log.info(
537         "VerificationNode: %d unsupported citation(s): %s",
538         len(unsupported),
539         unsupported,
540     )
541
542     rewrite_prompt = (
543         f"STUDENT QUESTION: {state.get('query', '')}\n\n"
544         f"RETRIEVED SOURCE MATERIAL:\n{format_context(state)}\n\n"
545         f"DRAFT ANSWER:\n{answer}\n\n"
546         f"UNSUPPORTED CITATIONS (must be removed or hedged):\n"
547         + "\n".join(f"- {c}" for c in unsupported)
548         + "\n\nRewrite the answer per the instructions. Return ONLY the rewritten answer."
549     )
550
551     try:
552         rewritten = llm_call(
553             rewrite_prompt,
554             model=SYNTHESIS_MODEL,
555             system_instruction=VERIFICATION_REWRITE_SYSTEM,
556         )
557         rewritten = (rewritten or "").strip()
558         if rewritten:
559             report["rewrites_applied"] = True
560             return {
561                 "final_answer": rewritten,
562                 "verification_report": report,
563                 "node_trace": _append_trace(state, "verification"),
564             }
565     except Exception as e: # graceful: leave answer unchanged on rewrite failure
566         log.warning("VerificationNode: rewrite failed (%s); keeping draft", e)
567
568     return {
569         "verification_report": report,
570         "node_trace": _append_trace(state, "verification"),
571     }
572
573
574 #

```

```

575 # ABLATION: MEGA-PROMPT (single LLM call - no multi-node graph)
576 #
577
578
579 MEGA_PROMPT_SYSTEM = (
580     "You are an expert Australian Property Law tutor. You will be given a student "
581     "question and retrieved source material. Produce a response with the sections below.\n\n"
582     # ■■■ Section 1: IRAC
583     "## SECTION 1 – IRAC ANALYSIS\n"
584     "Provide a full IRAC analysis structured with these exact headings:\n"
585     "- **Issue**: The legal question the court had to decide.\n"
586     "- **Rule**: The ratio decidendi – the binding legal principle.\n"
587     "- **Application**: How the court applied the rule to the facts.\n"
588     "- **Conclusion**: The court's decision.\n\n"
589     "Start this section with a one-sentence statement of the ratio decidendi on its own "
590     "line prefixed with 'Ratio Decidendi:\n\n"
591     # ■■■ Section 2: Mermaid diagram (conditional)
592     "## SECTION 2 – CHRONOLOGICAL FLOWCHART (conditional)\n"
593     "Evaluate the student's query. If the query asks for a timeline, "
594     "sequence of events, chain of events, flowchart, 'who did what', or 'what happened', "
595     "output a valid Mermaid.js `graph TD` diagram here.\n\n"
596     "When a diagram is required, follow these formatting rules:\n"
597     "- Use `graph TD` (top-down) direction.\n"
598     "- Node IDs must be camelCase (no spaces): e.g. `ownerDies`, `titlePasses`.\n"
599     "- Keep node labels SHORT – maximum 8-10 words. Put dates at the start.\n"
600     "  GOOD: `e1[\"1881: Clissold enters possession\"]`\n"
601     "  BAD: `e1[\"1881: Frederick Clissold enters into possession of the land\"]`\n"
602     "- Put extra detail on edge labels: `e1 -->|\"encloses land, pays rates\"| e2`\n"
603     "- Use curly brackets for decision nodes: `d1{\"Owner consents?\"}`\n"
604     "- Use subgraphs to group related phases.\n"
605     "- Wrap ALL labels in double quotes.\n"
606     "- Do NOT use the keyword `end` as a node ID.\n"
607     "- Aim for 5-12 nodes total.\n\n"
608     "Format a diagram as:\n"
609     "```mermaid\n<your flowchart>\n```\n\n"
610     # ■■■ Section 3: Summary
611     "## SECTION 3 – PLAIN-ENGLISH SUMMARY\n"
612     "Provide a concise plain-English explanation that answers the student's question "
613     "directly, drawing on the IRAC analysis and any diagram above. Use inline citations "
614     "where you rely on specific sources, e.g. (Source: Readings Week 3). Structure "
615     "your answer with headings and paragraphs.\n\n"
616     # ■■■ Grounding rules
617     "GROUNDING RULES (apply to all sections):\n"
618     "- ALL case names, dates, statutory references, legal tests, and holdings MUST "
619     "appear in the provided source material. Do NOT invent facts.\n"
620     "- Explanations and paraphrasing are encouraged – you are a tutor.\n"
621     "- If sources are insufficient, say so explicitly rather than filling gaps.\n"
622     "- If CONVERSATION HISTORY is provided, use it only to resolve pronouns and "
623     "follow-up references; all legal facts must still come from the sources."
624 )
625
626
627 def mega_prompt_node(state: AgentState) -> dict:
628     """Single-LLM ablation: retrieval context → one call produces IRAC + Mermaid + summary.
629
630     Bypasses the router and all specialised reasoning nodes. Used to measure
631     whether the multi-node LangGraph architecture adds value over a single
632     consolidated prompt.
633
634     Reads: query, retrieved_texts, retrieved_slides, chat_history
635     Writes: ratio_decidendi, irac_analysis, mermaid_diagram, chronology_summary,
636            final_answer, node_trace
637
638     """
639     query = state["query"]
640     context = _format_context(state)
641     history = get_chat_history(state)
642     hist_block = (
643         f"CONVERSATION HISTORY:\n{format_transcript_for_llm(history)}\n\n"
644         if history
645         else ""
646     )
647     prompt = (
648         f"{hist_block}"
649         f"STUDENT QUESTION: {query}\n\n"
650         f"SOURCE MATERIAL:\n{context}\n\n"
651         "Using only the source material above, produce all three sections as instructed."
652     )
653
654     log.info("MegaPromptNode: single consolidated LLM call (ablation)")
655     response = llm_call(prompt, model=ABLATION_MODEL, system_instruction=MEGA_PROMPT_SYSTEM)
656
657     # ■■■ Parse ratio decidendi

```

[illegible]

LangGraph state machine

[illegible]

[illegible]

src/eval/run_evals.py

Evaluation harness

```
1  """Evaluation and ablation framework for the Property Law Exam Assistant.
2
3  Compares the full agent pipeline against:
4      1. A plain LLM baseline (no retrieval, no graph nodes)
5      2. Mega-prompt ablation (retrieval + single consolidated LLM call replacing all nodes)
6
7  Eval cases are defined in ``EVAL_CASES``: each has a **query_family** tag
8  (`factual_retrieval`, `cross_modal_retrieval`, `analytical_synthesis`,
9  `conversational_followup`) so reports satisfy explicit per-family testing.
10 Multi-turn cases run the agent (and ablation) sequentially with memory; the
11 baseline always sees only the **final** user turn.
12
13 Metrics collected per run:
14     - Groundedness score (LLM-as-a-judge: 1-5 scale)
15     - Answer relevancy (LLM-as-a-judge: 1-5 scale)
16     - Context precision@K, MRR, Hit@K, NDCG@K (LLM-judged, binary relevance)
17     - Optional recall-vs-pool (`--retrieval-pool-eval`): expanded retrieve + judge
18     - Source diversity (unique sources retrieved)
19     - Total latency (seconds)
20     - Per-node latency breakdown (for full agent)
21     - Node trace (which nodes fired)
22     - Cross-modal retrieval diagnostics on slide-targeted cases
23
24 Results are exported as JSON and plots as PNG for the report.
25
26 Usage:
27     python -m src.eval.run_evals
28     python -m src.eval.run_evals --output-dir eval_results
29     python -m src.eval.run_evals --retrieval-pool-eval # optional recall-vs-pool (slow)
30 """
31
32 from __future__ import annotations
33
34 import argparse
35 import json
36 import logging
37 import math
38 import re
39 import time
40 from pathlib import Path
41 from typing import Literal, TypedDict
42
43 import matplotlib
44 matplotlib.use("Agg")
45 import matplotlib.pyplot as plt
46 from dotenv import load_dotenv
47
48 from src.agent.chat_memory import prepare_chat_history_for_run
49 from src.agent.nodes import (
50     _format_context,
51     chronology_node,
52     mega_prompt_node,
53     ratio_extractor_node,
54     retrieval_node,
55     router_node,
56     synthesis_node,
57     verification_node,
58 )
59 from src.agent.state import AgentState
60 from src.agent.tools import retrieve_all
61 from src.config import (
62     BASELINE_MODEL,
63     EVAL_RETRIEVAL_POOL_K_SLIDES,
64     EVAL_RETRIEVAL_POOL_K_TEXT,
65     JUDGE_CRITIQUE_MODEL,
66     JUDGE_DRAFT_MODEL,
67     USE_VERIFICATION,
68 )
69 from src.llm import get_token_usage, llm_call, reset_token_usage
70
71 load_dotenv()
72
73 logging.basicConfig(
74     level=logging.INFO,
75     format="%(asctime)s | %(levelname)-7s | %(message)s",
76     datefmt="%H:%M:%S",
77 )
```

[illegible]

[illegible]

```

244 {
245   "case_id": "conv_torrens_followup",
246   "query_family": "conversational_followup",
247   "rationale": "Contrast question after Torrens explanation – coreference to prior answer.",
248   "user_turns": [
249     "Explain the Torrens system of land registration.",
250     "In one paragraph, how does that differ from a deeds registry? Stay within "
251     "our indexed course sources only.",
252   ],
253 },
254 {
255   "case_id": "conv_chattels_found",
256   "query_family": "conversational_followup",
257   "rationale": "Hypothetical extension after test – requires prior topic + retrieval.",
258   "user_turns": [
259     "What is the legal test for establishing possession of chattels?",
260     "Does anything in our indexed materials suggest the answer changes if the "
261     "chattel was found on the ground rather than handed over? Explain briefly.",
262   ],
263 },
264 # ■■ Expansion (v2): seven additional cases for statistical power ■■■■■■■■■■
265 {
266   "case_id": "fact_fee_tail_vs_simple",
267   "query_family": "factual_retrieval",
268   "rationale": "Comparative doctrine question – should retrieve estate-type readings.",
269   "user_turns": [
270     "What is the difference between a fee tail and a fee simple in our materials?",
271   ],
272 },
273 {
274   "case_id": "fact_pla_chattels",
275   "query_family": "factual_retrieval",
276   "rationale": "Statute lookup – Property Law Act 1974 (Qld) on chattels.",
277   "user_turns": [
278     "What does the Property Law Act 1974 (Qld) say about chattels in the readings?",
279   ],
280 },
281 {
282   "case_id": "xmodal_torrens_indefeasibility",
283   "query_family": "cross_modal_retrieval",
284   "rationale": "Targets lecture slides on Torrens indefeasibility – VLM-described content.",
285   "user_turns": [
286     "Looking at the indexed lecture slides on the Torrens system, how do those "
287     "materials present the doctrine of indefeasibility of title?",
288   ],
289 },
290 {
291   "case_id": "xmodal_bailment_categories",
292   "query_family": "cross_modal_retrieval",
293   "rationale": "Slide-anchored taxonomy: bailment categories from lecture deck.",
294   "user_turns": [
295     "From the lecture slides indexed in the knowledge base, what categories "
296     "of bailment do those materials describe?",
297   ],
298 },
299 {
300   "case_id": "anal_legal_vs_equitable_interests",
301   "query_family": "analytical_synthesis",
302   "rationale": "Comparison across multiple chunks of equitable vs legal interests.",
303   "user_turns": [
304     "Using multiple chunks of the indexed sources, compare equitable interests "
305     "with legal interests in property: how do they arise, and how do they bind "
306     "third parties?",
307   ],
308 },
309 {
310   "case_id": "anal_torrens_title_passing",
311   "query_family": "analytical_synthesis",
312   "rationale": "Sequenced reasoning: passage of legal title in a Torrens transfer.",
313   "user_turns": [
314     "Sequence the steps by which legal title passes in a Torrens-system land "
315     "transfer, citing the indexed course materials at each step.",
316   ],
317 },
318 {
319   "case_id": "conv_lease_three_turn",
320   "query_family": "conversational_followup",
321   "rationale": "Three-turn case where the third turn references the second.",
322   "user_turns": [
323     "What are the essential characteristics of a lease in our indexed materials?",
324     "Now contrast a lease with a licence in one paragraph, again from the sources.",
325     "Which of those points you just made would change if the licence were "
326     "irrevocable? Stay within our indexed sources only.",

```

[illegible]

[illegible]

[illegible]

```

576     """Judge each retrieved chunk for relevance.
577
578     Returns precision@K, MRR, Hit@K, NDCG@K (binary labels), and per-chunk verdicts.
579     """
580     if not chunks:
581         return {
582             "precision_at_k": 0.0,
583             "relevant_count": 0,
584             "total_count": 0,
585             "verdicts": [],
586             **_ranking_metrics_from_binary_verdicts([]),
587         }
588
589     verdicts: list[bool] = []
590     for chunk in chunks:
591         prompt = (
592             f"QUESTION: {query}\n\n"
593             f"RETRIEVED CHUNK (from {chunk.get('source', 'unknown')}):\n"
594             f"{chunk.get('content', '')[:1500]}\n\n"
595             "Is this chunk relevant to the question?"
596         )
597         result = _judge_call(prompt, JUDGE_CONTEXT_PRECISION_SYSTEM)
598         verdicts.append(bool(result.get("relevant", False)))
599
600     relevant_count = sum(verdicts)
601     ranking = _ranking_metrics_from_binary_verdicts(verdicts)
602     return {
603         "precision_at_k": round(relevant_count / len(chunks), 2),
604         "relevant_count": relevant_count,
605         "total_count": len(chunks),
606         "verdicts": verdicts,
607         **ranking,
608     }
609
610
611 def judge_context_precision_with_pool(
612     query: str,
613     *,
614     week: str | None = None,
615     k_text_prod: int = 8,
616     k_slide_prod: int = 4,
617     k_text_pool: int = EVAL_RETRIEVAL_POOL_K_TEXT,
618     k_slide_pool: int = EVAL_RETRIEVAL_POOL_K_SLIDES,
619 ) -> dict:
620     """LLM-judge a **deep** retrieval pool, then score production slice + recall estimate.
621
622     **Recall vs pool:** ``relevant_in_production / max(relevant_in_pool, 1)`` – treats
623     all judge-positive chunks in the pool as a proxy for the relevant set. This is
624     **not** corpus-wide recall (no human qrels); document that limitation in the report.
625
626     Ranking metrics (MRR, NDCG, Hit) use **production** ordering (first *k_text_prod*
627     text chunks + first *k_slide_prod* slides from the same ranked lists).
628     """
629     texts, slides = retrieve_all(
630         query,
631         week=week,
632         k_text=k_text_pool,
633         k_slides=k_slide_pool,
634     )
635     pool_chunks = list(texts) + list(slides)
636     base = judge_context_precision(query, pool_chunks)
637     verdicts: list[bool] = base["verdicts"]
638     n_t = len(texts)
639     n_s = len(slides)
640     prod_indices = list(range(min(k_text_prod, n_t))) + list(
641         range(n_t, n_t + min(k_slide_prod, n_s))
642     )
643     prod_verdicts = [verdicts[i] for i in prod_indices if i < len(verdicts)]
644     rel_prod = sum(prod_verdicts)
645     total_rel_pool = sum(verdicts)
646     recall_vs_pool: float | None = None
647     if total_rel_pool > 0:
648         recall_vs_pool = round(rel_prod / total_rel_pool, 3)
649
650     rank_prod = _ranking_metrics_from_binary_verdicts(prod_verdicts)
651
652     return {
653         "precision_at_k": round(rel_prod / max(len(prod_verdicts), 1), 2),
654         "relevant_count": int(rel_prod),
655         "total_count": len(prod_verdicts),
656         "verdicts": prod_verdicts,
657         **rank_prod,
658         "recall_vs_pool": recall_vs_pool,

```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

1073     ablation_no_rr_ground = judge_groundedness(
1074         judge_q, ablation_no_rr_context, ablation_no_rr_result["answer"]
1075     )
1076
1077     # 6. Judge answer relevancy
1078     log.info("Judging answer relevancy...")
1079     agent_relevancy = judge_answer_relevancy(judge_q, agent_result["answer"])
1080     baseline_relevancy = judge_answer_relevancy(judge_q, baseline_result["answer"])
1081     ablation_relevancy = judge_answer_relevancy(judge_q, ablation_result["answer"])
1082     ablation_no_rr_relevancy = judge_answer_relevancy(
1083         judge_q, ablation_no_rr_result["answer"]
1084     )
1085
1086     # 7. Retrieval metrics (relevance of chunks to the **final** user utterance)
1087     if retrieval_pool_eval:
1088         log.info("Judging retrieval (expanded pool - slow)...")
1089         agent_precision = judge_context_precision_with_pool(last_q, week=None)
1090         ablation_precision = judge_context_precision_with_pool(last_q, week=None)
1091         ablation_no_rr_precision = judge_context_precision_with_pool(last_q, week=None)
1092     else:
1093         log.info("Judging context precision + ranking (production chunks)...")
1094         agent_precision = judge_context_precision(last_q, agent_chunks)
1095         ablation_precision = judge_context_precision(last_q, ablation_chunks)
1096         ablation_no_rr_precision = judge_context_precision(last_q, ablation_no_rr_chunks)
1097
1098     # 8. Mermaid validity and IRAC compliance (hypothesis-specific metrics)
1099     # Use dedicated node artefacts when present; otherwise score embedded content in
1100     # final_answer so ablation/baseline are comparable to the full agent.
1101     agent_mermaid = check_mermaid_validity(
1102         _mermaid_for_structural_metric(
1103             agent_result.get("mermaid_diagram", ""),
1104             agent_result.get("answer", ""),
1105         )
1106     )
1107     ablation_mermaid = check_mermaid_validity(
1108         _mermaid_for_structural_metric(
1109             ablation_result.get("mermaid_diagram", ""),
1110             ablation_result.get("answer", ""),
1111         )
1112     )
1113     baseline_mermaid = check_mermaid_validity(
1114         _mermaid_for_structural_metric("", baseline_result.get("answer", ""))
1115     )
1116
1117     agent_intent = agent_result.get("intent")
1118     ablation_no_rr_intent = ablation_no_rr_result.get("intent")
1119     # Mega-prompt and baseline have no router; treat as IRAC-applicable
1120     # (they always produce a single legal answer, no chronology branch).
1121     agent_irac = check_irac_compliance(
1122         _irac_for_structural_metric(
1123             agent_result.get("irac_analysis", ""),
1124             agent_result.get("answer", ""),
1125         ),
1126         intent=agent_intent,
1127     )
1128     ablation_irac = check_irac_compliance(
1129         _irac_for_structural_metric(
1130             ablation_result.get("irac_analysis", ""),
1131             ablation_result.get("answer", ""),
1132         )
1133     )
1134     baseline_irac = check_irac_compliance(
1135         _irac_for_structural_metric("", baseline_result.get("answer", ""))
1136     )
1137
1138     ablation_no_rr_mermaid = check_mermaid_validity(
1139         _mermaid_for_structural_metric(
1140             ablation_no_rr_result.get("mermaid_diagram", ""),
1141             ablation_no_rr_result.get("answer", ""),
1142         )
1143     )
1144     ablation_no_rr_irac = check_irac_compliance(
1145         _irac_for_structural_metric(
1146             ablation_no_rr_result.get("irac_analysis", ""),
1147             ablation_no_rr_result.get("answer", ""),
1148         ),
1149         intent=ablation_no_rr_intent,
1150     )
1151
1152     def _fmt_pct(d: dict) -> str:
1153         s = d.get("score")
1154         return "n/a" if s is None else f"{s * 100:.0f}%"
1155

```

```

1156     log.info(
1157         "Structural – Mermaid: Agent=%s | Ablation=%s | Baseline=%s | "
1158         "IRAC: Agent=%s | Ablation=%s | Baseline=%s",
1159         _fmt_pct(agent_mermaid),
1160         _fmt_pct(ablation_mermaid),
1161         _fmt_pct(baseline_mermaid),
1162         _fmt_pct(agent_irac),
1163         _fmt_pct(ablation_irac),
1164         _fmt_pct(baseline_irac),
1165     )
1166
1167     cross_modal_block: dict | None = None
1168     if case["query_family"] == "cross_modal_retrieval":
1169         cross_modal_block = _cross_modal_signals(agent_result)
1170
1171     result_entry = {
1172         "query_family": case["query_family"],
1173         "case_id": case["case_id"],
1174         "case_rationale": case["rationale"],
1175         "query": last_q,
1176         "judge_question": judge_q,
1177         "user_turns": list(case["user_turns"]),
1178         "agent": {
1179             **agent_result,
1180             "groundedness": agent_ground,
1181             "answer_relevancy": agent_relevancy,
1182             "context_precision": agent_precision,
1183             "mermaid_validity": agent_mermaid,
1184             "irac_compliance": agent_irac,
1185         },
1186         "baseline": {
1187             **baseline_result,
1188             "groundedness": baseline_ground,
1189             "answer_relevancy": baseline_relevancy,
1190             "mermaid_validity": baseline_mermaid,
1191             "irac_compliance": baseline_irac,
1192         },
1193         "ablation_mega_prompt": {
1194             **ablation_result,
1195             "groundedness": ablation_ground,
1196             "answer_relevancy": ablation_relevancy,
1197             "context_precision": ablation_precision,
1198             "mermaid_validity": ablation_mermaid,
1199             "irac_compliance": ablation_irac,
1200         },
1201         "ablation_no_reranker": {
1202             **ablation_no_rr_result,
1203             "groundedness": ablation_no_rr_ground,
1204             "answer_relevancy": ablation_no_rr_relevancy,
1205             "context_precision": ablation_no_rr_precision,
1206             "mermaid_validity": ablation_no_rr_mermaid,
1207             "irac_compliance": ablation_no_rr_irac,
1208         },
1209     }
1210     if cross_modal_block is not None:
1211         result_entry["cross_modal_signals"] = cross_modal_block
1212
1213     all_results.append(result_entry)
1214     log.info(
1215         "Groundedness – Agent: %d, Baseline: %d, Ablated: %d | "
1216         "Relevancy – Agent: %d, Baseline: %d, Ablated: %d | "
1217         "P@K %.2f / MRR %.3f / NDCG %.3f (agent) | "
1218         "P@K %.2f / MRR %.3f / NDCG %.3f (ablation)",
1219         agent_ground["score"], baseline_ground["score"], ablation_ground["score"],
1220         agent_relevancy["score"], baseline_relevancy["score"], ablation_relevancy["score"],
1221         agent_precision["precision_at_k"],
1222         agent_precision.get("mrr", 0.0),
1223         agent_precision.get("ndcg_at_k", 0.0),
1224         ablation_precision["precision_at_k"],
1225         ablation_precision.get("mrr", 0.0),
1226         ablation_precision.get("ndcg_at_k", 0.0),
1227     )
1228
1229     # Save raw results
1230     results_path = output_dir / "eval_results.json"
1231     with open(results_path, "w") as f:
1232         json.dump(all_results, f, indent=2, default=str)
1233     log.info("Results saved to %s", results_path)
1234
1235     # Generate summary
1236     summary = _compute_summary(all_results)
1237     summary_path = output_dir / "eval_summary.json"
1238     with open(summary_path, "w") as f:

```

```

1239         json.dump(summary, f, indent=2)
1240     log.info("Summary saved to %s", summary_path)
1241
1242     # Generate plots
1243     _generate_plots(all_results, summary, output_dir)
1244     log.info("Plots saved to %s", output_dir)
1245
1246     # Generate failure analysis
1247     _generate_failure_analysis(all_results, output_dir)
1248
1249     # Emit human spot-check scaffolding (qualitative validation of LLM judges)
1250     _human_spot_check_log(all_results, output_dir)
1251
1252     return summary
1253
1254
1255 def _compute_summary(results: list[dict]) -> dict:
1256     """Compute aggregate metrics from evaluation results."""
1257     # Include the new `ablation_no_reranker` configuration when present.
1258     base_configs = ["agent", "baseline", "ablation_mega_prompt"]
1259     if results and "ablation_no_reranker" in results[0]:
1260         configs = base_configs + ["ablation_no_reranker"]
1261     else:
1262         configs = base_configs
1263     summary = {}
1264
1265     for config in configs:
1266         scores = [r[config]["groundedness"]["score"] for r in results]
1267         relevancies = [r[config]["answer_relevancy"]["score"] for r in results]
1268         latencies = [r[config]["latency_s"] for r in results]
1269         diversities = [r[config].get("source_diversity", 0) for r in results]
1270
1271         mermaid_scores = [r[config]["mermaid_validity"]["score"] for r in results]
1272         # IRAC scores may be None when the router intent is "chronology" or
1273         # "general" (IRAC structure is not expected). Exclude those from the
1274         # average and report counts separately.
1275         irac_scores_raw = [r[config]["irac_compliance"]["score"] for r in results]
1276         irac_scores_applicable = [s for s in irac_scores_raw if s is not None]
1277         irac_avg = (
1278             round(sum(irac_scores_applicable) / len(irac_scores_applicable), 2)
1279             if irac_scores_applicable
1280             else 0.0
1281         )
1282
1283     entry = {
1284         "avg_groundedness": round(sum(scores) / len(scores), 2),
1285         "avg_answer_relevancy": round(sum(relevancies) / len(relevancies), 2),
1286         "avg_mermaid_validity": round(sum(mermaid_scores) / len(mermaid_scores), 2),
1287         "avg_irac_compliance": irac_avg,
1288         "irac_applicable_count": len(irac_scores_applicable),
1289         "irac_not_applicable_count": len(irac_scores_raw) - len(irac_scores_applicable),
1290         "avg_latency_s": round(sum(latencies) / len(latencies), 2),
1291         "avg_source_diversity": round(sum(diversities) / len(diversities), 2),
1292         "groundedness_scores": scores,
1293         "answer_relevancy_scores": relevancies,
1294         "mermaid_scores": mermaid_scores,
1295         "irac_scores": irac_scores_raw,
1296         "latencies": latencies,
1297     }
1298
1299     # Context precision (agent and ablation only)
1300     if "context_precision" in results[0].get(config, {}):
1301         precisions = [r[config]["context_precision"]["precision_at_k"] for r in results]
1302         entry["avg_context_precision"] = round(sum(precisions) / len(precisions), 2)
1303         entry["context_precisions"] = precisions
1304         cp0 = results[0][config]["context_precision"]
1305         if "mrr" in cp0:
1306             entry["avg_mrr"] = round(
1307                 sum(r[config]["context_precision"]["mrr"] for r in results) / len(results),
1308                 3,
1309             )
1310             entry["avg_hit_at_k"] = round(
1311                 sum(r[config]["context_precision"]["hit_at_k"] for r in results) / len(results),
1312                 3,
1313             )
1314             entry["avg_ndcg_at_k"] = round(
1315                 sum(r[config]["context_precision"]["ndcg_at_k"] for r in results) / len(results),
1316                 3,
1317             )
1318             entry["mrr_per_query"] = [r[config]["context_precision"]["mrr"] for r in results]
1319             entry["hit_at_k_per_query"] = [
1320                 r[config]["context_precision"]["hit_at_k"] for r in results
1321             ]

```

[illegible]

```

def _generate_failure_analysis(results: list[dict], output_dir: Path) -> None:
    """Analyse evaluation results and generate a failure analysis report."""
    lines = list[str] = [
        "# Failure Analysis Report",
        "",
        "Auto-generated from evaluation results. This report identifies queries where "
        "the system underperformed and categorises the root causes.",
        ""
    ]

    # ■■■ 0. Explicit query family coverage (coursework requirement) ■■■■■■■■■■
    lines.append("## 0. Query family coverage")
    lines.append("")
    lines.append(
        "Every eval case is assigned **exactly one** of four families. "
        "Use this table for success/failure discussion per family."
    )

    lines.append("")
    lines.append("| Family | Cases | Case IDs |")
    lines.append("|-----|-----|-----|")
    fam_order = [
        "factual_retrieval",
        "cross_modal_retrieval",
        "analytical_synthesis",
        "conversational_followup",
    ]
    by_fam: dict[str, list[dict]] = {k: [] for k in fam_order}
    for r in results:
        fam = r.get("query_family", "unknown")
        by_fam.setdefault(fam, []).append(r)
    for fam in fam_order + [k for k in sorted(by_fam.keys()) if k not in fam_order]:
        rows = by_fam.get(fam, [])
        if not rows:
            continue
        ids = ", ".join(str(x.get("case_id", "?")) for x in rows)
        lines.append(f"| **{fam}** | {len(rows)} | {ids} |")
    lines.append("")
    for fam in fam_order:
        desc = QUERY_FAMILY_DESCRIPTIONS.get(fam)
        if desc:
            lines.append(f"- **{fam}:** {desc}")
    lines.append("")

    for fam in fam_order:
        rows = by_fam.get(fam, [])
        if not rows:
            continue
        lines.append(f"### Family: `{fam}`")
        lines.append("")
        for r in rows:
            cid = r.get("case_id", "?")
            ag = r["agent"]["groundedness"]["score"]
            ar = r["agent"]["answer_relevancy"]["score"]
            turns = r.get("user_turns", [r.get("query", "")])
            turn_note = f"{len(turns)} turn(s)" if len(turns) > 1 else "single turn"
            lines.append(f"- **{cid}** ({turn_note}): groundedness **{ag}/5**, relevancy **{ar}/5**")
            rat = r.get("case_rationale")
            if rat:
                lines.append(f" - *Rationale:* {rat}")
            xm = r.get("cross_modal_signals")
            if xm:
                lines.append(
                    f" - *Retrieval:* text chunks={xm.get('retrieved_text_chunks', 0)}, "
                    f"slide chunks={xm.get('retrieved_slide_chunks', 0)}, "
                    f"both_modalities={xm.get('both_modalities_retrieved')}"
                )
            lines.append("")

    # ■■■ 1. Low groundedness (agent scored < 4) ■■■■■■■■■■
    low_ground = [
        r for r in results
        if r["agent"]["groundedness"]["score"] < 4
    ]
    lines.append("## 1. Low Groundedness (Agent < 4/5)")
    lines.append("")
    if low_ground:
        for r in low_ground:
            score = r["agent"]["groundedness"]["score"]
            reasoning = r["agent"]["groundedness"]["reasoning"]
            fam = r.get("query_family", "N/A")
            cid = r.get("case_id", "N/A")
            lines.append(f"### [{fam}] / {cid} \ {r['query']}\n")
            lines.append(f"- **Score:** {score}/5")

```

[illegible]

[illegible]

```

1650     if has_no_rr:
1651         configs.append("ablation_no_reranker")
1652         labels.append("Agent\n(no reranker)")
1653         colors.append("#10b981")
1654
1655     # 1. Groundedness comparison
1656     fig, ax = plt.subplots(figsize=(8, 5))
1657     scores = [summary[c]["avg_groundedness"] for c in configs]
1658     bars = ax.bar(labels, scores, color=colors, edgecolor="black")
1659     ax.set_ylabel("Average Groundedness Score (1-5)")
1660     ax.set_title("Groundedness: Full Agent vs Baseline vs Ablation")
1661     ax.set_ylim(0, 5.5)
1662     for bar, score in zip(bars, scores):
1663         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.1,
1664                 f"{score:.1f}", ha="center", fontweight="bold")
1665     plt.tight_layout()
1666     plt.savefig(output_dir / "groundedness_comparison.png", dpi=150)
1667     plt.close()
1668
1669     # 2. Answer relevancy comparison
1670     fig, ax = plt.subplots(figsize=(8, 5))
1671     relevancies = [summary[c]["avg_answer_relevancy"] for c in configs]
1672     bars = ax.bar(labels, relevancies, color=colors, edgecolor="black")
1673     ax.set_ylabel("Average Answer Relevancy (1-5)")
1674     ax.set_title("Answer Relevancy: Full Agent vs Baseline vs Ablation")
1675     ax.set_ylim(0, 5.5)
1676     for bar, rel in zip(bars, relevancies):
1677         ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.1,
1678                 f"{rel:.1f}", ha="center", fontweight="bold")
1679     plt.tight_layout()
1680     plt.savefig(output_dir / "answer_relevancy_comparison.png", dpi=150)
1681     plt.close()
1682
1683     # 3. Context precision comparison (agent and ablation only)
1684     retrieval_configs = ["agent", "ablation_mega_prompt"]
1685     retrieval_labels = ["Full Agent\n(MMR)", "Mega-Prompt\n(Ablation)"]
1686     precisions = [summary[c].get("avg_context_precision", 0) for c in retrieval_configs]
1687     if any(p > 0 for p in precisions):
1688         fig, ax = plt.subplots(figsize=(8, 5))
1689         bars = ax.bar(retrieval_labels, precisions, color=["#2563eb", "#f59e0b"], edgecolor="black")
1690         ax.set_ylabel("Average Context Precision@K")
1691         ax.set_title("Retrieval Quality: Context Precision@K")
1692         ax.set_ylim(0, 1.1)
1693         for bar, prec in zip(bars, precisions):
1694             ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.02,
1695                     f"{prec:.2f}", ha="center", fontweight="bold")
1696         plt.tight_layout()
1697         plt.savefig(output_dir / "context_precision.png", dpi=150)
1698         plt.close()
1699
1700     # 3b. Ranking metrics (agent vs ablation)
1701     if all("avg_mrr" in summary[c] for c in retrieval_configs):
1702         fig, ax = plt.subplots(figsize=(9, 5))
1703         metrics_rank = ["MRR", "Hit@K", "NDCG@K"]
1704         xw = range(len(metrics_rank))
1705         width = 0.35
1706         agent_vals = [
1707             summary["agent"]["avg_mrr"],
1708             summary["agent"]["avg_hit_at_k"],
1709             summary["agent"]["avg_ndcg_at_k"],
1710         ]
1711         ab_vals = [
1712             summary["ablation_mega_prompt"]["avg_mrr"],
1713             summary["ablation_mega_prompt"]["avg_hit_at_k"],
1714             summary["ablation_mega_prompt"]["avg_ndcg_at_k"],
1715         ]
1716         ax.bar([i - width / 2 for i in xw], agent_vals, width, label="Full Agent", color="#2563eb", edgecolor="black")
1717         ax.bar([i + width / 2 for i in xw], ab_vals, width, label="Ablation", color="#f59e0b", edgecolor="black")
1718         ax.set_xticks(list(xw))
1719         ax.set_xticklabels(metrics_rank)
1720         ax.set_ylabel("Score (0-1)")
1721         ax.set_title("Retrieval ranking metrics (LLM-judged binary relevance)")
1722         ax.set_ylim(0, 1.1)
1723         ax.legend()
1724         plt.tight_layout()
1725         plt.savefig(output_dir / "retrieval_ranking_metrics.png", dpi=150)
1726         plt.close()
1727
1728     # 4. Latency comparison
1729     fig, ax = plt.subplots(figsize=(8, 5))
1730     latencies = [summary[c]["avg_latency_s"] for c in configs]

```

```

1731 bars = ax.bar(labels, latencies, color=colors, edgecolor="black")
1732 ax.set_ylabel("Average Latency (seconds)")
1733 ax.set_title("Latency: Full Agent vs Baseline vs Ablation")
1734 for bar, lat in zip(bars, latencies):
1735     ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,
1736            f"{lat:.1f}s", ha="center", fontweight="bold")
1737 plt.tight_layout()
1738 plt.savefig(output_dir / "latency_comparison.png", dpi=150)
1739 plt.close()
1740
1741 # 5. Per-query groundedness breakdown
1742 fig, ax = plt.subplots(figsize=(12, 6))
1743 x = range(len(results))
1744 query_labels = [f"Q{i+1}" for i in x]
1745
1746 short_labels = ["Agent", "Baseline", "Ablation"] + (["No-Reranker"] if has_no_rr else [])
1747 bar_w = 0.22 if has_no_rr else 0.25
1748 n = len(configs)
1749 for i, (config, label) in enumerate(zip(configs, short_labels)):
1750     scores = [r[config]["groundedness"]["score"] for r in results]
1751     offset = (i - (n - 1) / 2) * bar_w
1752     ax.bar([xi + offset for xi in x], scores, width=bar_w, label=label)
1753
1754 ax.set_ylabel("Groundedness Score")
1755 ax.set_title("Per-Query Groundedness Breakdown")
1756 ax.set_xticks(list(x))
1757 ax.set_xticklabels(query_labels)
1758 ax.set_ylim(0, 5.5)
1759 ax.legend()
1760 plt.tight_layout()
1761 plt.savefig(output_dir / "per_query_groundness.png", dpi=150)
1762 plt.close()
1763
1764 # 6. Source diversity comparison
1765 fig, ax = plt.subplots(figsize=(8, 5))
1766 diversities = [summary[c]["avg_source_diversity"] for c in configs]
1767 bars = ax.bar(labels, diversities, color=colors, edgecolor="black")
1768 ax.set_ylabel("Average Unique Sources Retrieved")
1769 ax.set_title("Source Diversity: Full Agent vs Baseline vs Ablation")
1770 for bar, div in zip(bars, diversities):
1771     ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.1,
1772            f"{div:.1f}", ha="center", fontweight="bold")
1773 plt.tight_layout()
1774 plt.savefig(output_dir / "source_diversity.png", dpi=150)
1775 plt.close()
1776
1777 # 7. Per-node latency breakdown
1778 node_lats = summary["agent"].get("avg_node_latencies", {})
1779 if node_lats:
1780     fig, ax = plt.subplots(figsize=(8, 5))
1781     nodes = list(node_lats.keys())
1782     values = list(node_lats.values())
1783     bars = ax.barh(nodes, values, color="#2563eb", edgecolor="black")
1784     ax.set_xlabel("Average Latency (seconds)")
1785     ax.set_title("Per-Node Latency Breakdown (Full Agent)")
1786     for bar, val in zip(bars, values):
1787         ax.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height() / 2,
1788                f"{val:.1f}s", va="center", fontweight="bold")
1789     plt.tight_layout()
1790     plt.savefig(output_dir / "node_latency_breakdown.png", dpi=150)
1791     plt.close()
1792
1793 # 8. Radar / multi-metric summary
1794 fig, ax = plt.subplots(figsize=(10, 5))
1795 metric_names = ["Groundedness", "Answer\nRelevancy", "Source\nDiversity\n(norm)"]
1796 radar_labels = ["Full Agent", "Baseline", "Ablation"] + (["No-Reranker"] if has_no_rr else [])
1797 for config, label, color in zip(configs, radar_labels, colors):
1798     vals = [
1799         summary[config]["avg_groundness"],
1800         summary[config]["avg_answer_relevancy"],
1801         min(summary[config]["avg_source_diversity"] / 8.0 * 5, 5.0),
1802     ]
1803     if "avg_context_precision" in summary[config]:
1804         metric_names_ext = metric_names + ["Context\nPrecision"]
1805         vals.append(summary[config]["avg_context_precision"] * 5)
1806     else:
1807         metric_names_ext = metric_names
1808     x_pos = range(len(vals))
1809     ax.plot(list(x_pos), vals, "o-", label=label, color=color, linewidth=2, markersize=8)
1810
1811 ax.set_xticks(range(len(metric_names_ext)))
1812 ax.set_xticklabels(metric_names_ext)
1813 ax.set_ylim(0, 5.5)

```

```

1814     ax.set_ylabel("Score (normalised to 0-5)")
1815     ax.set_title("Multi-Metric Comparison")
1816     ax.legend()
1817     ax.grid(axis="y", alpha=0.3)
1818     plt.tight_layout()
1819     plt.savefig(output_dir / "multi_metric_comparison.png", dpi=150)
1820     plt.close()
1821
1822     # 9. Token usage comparison (stacked: input vs output)
1823     if all("total_tokens" in summary[c] for c in configs):
1824         fig, ax = plt.subplots(figsize=(8, 5))
1825         input_tokens = [summary[c]["total_input_tokens"] for c in configs]
1826         output_tokens = [summary[c]["total_output_tokens"] for c in configs]
1827         bars1 = ax.bar(labels, input_tokens, label="Input Tokens", color="#2563eb", edgecolor="black")
1828         bars2 = ax.bar(labels, output_tokens, bottom=input_tokens, label="Output Tokens", color="#60a5fa", edg
ecolor="black")
1829         ax.set_ylabel("Total Tokens (across all queries)")
1830         ax.set_title("Token Usage: Full Agent vs Baseline vs Ablation")
1831         ax.legend()
1832         for bar, inp, out in zip(bars1, input_tokens, output_tokens):
1833             total = inp + out
1834             ax.text(bar.get_x() + bar.get_width() / 2, total + max(input_tokens) * 0.02,
1835                     f"{total:,}", ha="center", fontweight="bold", fontsize=9)
1836         plt.tight_layout()
1837         plt.savefig(output_dir / "token_usage_comparison.png", dpi=150)
1838         plt.close()
1839
1840     # 10. Structural metrics: Mermaid validity + IRAC compliance
1841     fig, ax = plt.subplots(figsize=(10, 5))
1842     x = range(len(configs))
1843     width = 0.35
1844     mermaid_vals = [summary[c]["avg_mermaid_validity"] * 100 for c in configs]
1845     irac_vals = [summary[c]["avg_irac_compliance"] * 100 for c in configs]
1846
1847     bars1 = ax.bar([xi - width / 2 for xi in x], mermaid_vals, width, label="Mermaid Validity", color="#2563eb
", edgecolor="black")
1848     bars2 = ax.bar([xi + width / 2 for xi in x], irac_vals, width, label="IRAC Compliance", color="#f59e0b", e
dgecolor="black")
1849     ax.set_ylabel("Score (%)")
1850     ax.set_title("Structural Output Quality: Mermaid Validity & IRAC Compliance")
1851     ax.set_xticks(list(x))
1852     ax.set_xticklabels(labels)
1853     ax.set_ylim(0, 110)
1854     ax.legend()
1855     for bar, val in zip(list(bars1) + list(bars2), mermaid_vals + irac_vals):
1856         if val > 0:
1857             ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 2,
1858                     f"{val:.0f}%", ha="center", fontweight="bold", fontsize=9)
1859         else:
1860             ax.text(bar.get_x() + bar.get_width() / 2, 2,
1861                     "0%", ha="center", fontweight="bold", fontsize=9, color="red")
1862     plt.tight_layout()
1863     plt.savefig(output_dir / "structural_metrics.png", dpi=150)
1864     plt.close()
1865
1866     # 11. Agent groundedness by query family
1867     bf = summary.get("by_query_family")
1868     if bf:
1869         fam_keys = list(bf.keys())
1870         fig, ax = plt.subplots(figsize=(9, 5))
1871         scores = [bf[k]["agent_avg_groundedness"] for k in fam_keys]
1872         colours = ["#2563eb", "#7c3aed", "#059669", "#ea580c"]
1873         bar_cols = [colours[i % len(colours)] for i in range(len(fam_keys))]
1874         bars = ax.bar(fam_keys, scores, color=bar_cols, edgecolor="black")
1875         ax.set_ylabel("Average groundedness (1-5)")
1876         ax.set_title("Full agent: groundedness by query family")
1877         ax.set_ylim(0, 5.5)
1878         ax.tick_params(axis="x", rotation=15)
1879         for bar, score in zip(bars, scores):
1880             ax.text(
1881                 bar.get_x() + bar.get_width() / 2,
1882                 bar.get_height() + 0.08,
1883                 f"{score:.2f}",
1884                 ha="center",
1885                 fontweight="bold",
1886                 fontsize=9,
1887             )
1888         plt.tight_layout()
1889         plt.savefig(output_dir / "groundedness_by_query_family.png", dpi=150)
1890         plt.close()
1891
1892     # 12. Precision: agent (with reranker) vs ablation_no_reranker
1893     if has_no_rr:

```

[illegible]

[illegible]

```

2060     print(f"    Avg Context Precision: {metrics['avg_context_precision']}")
2061     retrieval_configs = ("agent", "ablation_mega_prompt", "ablation_no_reranker")
2062     if config in retrieval_configs and "avg_mrr" in metrics:
2063         print(f"    Avg MRR / Hit@K / NDCG: {metrics['avg_mrr']} / {metrics['avg_hit_at_k']} / {metrics['a
vg_ndcg_at_k']}")
2064     if config in retrieval_configs and metrics.get("avg_recall_vs_pool") is not None:
2065         print(f"    Avg recall vs pool: {metrics['avg_recall_vs_pool']}")
2066     print(f"    Avg Latency: {metrics['avg_latency_s']}s")
2067     print(f"    Avg Source Diversity: {metrics['avg_source_diversity']}")
2068     if "total_tokens" in metrics:
2069         print(f"    Total Tokens: {metrics['total_tokens']:,}")
2070     print(f"    Avg Tokens/Query: {metrics['avg_tokens_per_query']:,}")
2071     if "avg_node_latencies" in metrics:
2072         print(f"    Node Latencies: {metrics['avg_node_latencies']}")
2073
2074     if by_fam:
2075         print("\n By query family (full agent):")
2076         for fam, block in by_fam.items():
2077             print(f"    {fam}: groundedness {block['agent_avg_groundedness']/5}, "
2078                   f"relevancy {block['agent_avg_answer_relevancy']/5}, "
2079                   f"n={block['case_count']}")
2080             if block.get("fraction_cases_with_text_and_slides_retrieved") is not None:
2081                 print(
2082                     "    cross-modal both text+slides retrieved: "
2083                     f"{block['fraction_cases_with_text_and_slides_retrieved']:.0%} of cases"
2084                 )
2085
2086
2087 if __name__ == "__main__":
2088     main()

```

Streamlit frontend

[illegible]

```

78 if "messages" not in st.session_state:
79     st.session_state.messages = []
80 
81 st.title("Property Law Exam Assistant")
82 
83 for msg in st.session_state.messages:
84     with st.chat_message(msg["role"]):
85         render_message(msg["content"])
86 
87 # ■■ Chat Input ████████████████████████████████████████████████████████████
88 
89 prompt = st.chat_input("Ask a property law question...")
90 
91 if prompt:
92     chat_prior = [dict(m) for m in st.session_state.messages]
93     st.session_state.messages.append({"role": "user", "content": prompt})
94     with st.chat_message("user"):
95         st.markdown(prompt)
96 
97     with st.chat_message("assistant"):
98         with st.spinner("Thinking..."):
99             start = time.time()
100             result = run_query(
101                 prompt,
102                 week_filter=week_filter,
103                 chat_history=chat_prior or None,
104             )
105             elapsed = time.time() - start
106 
107     # ■■ Intent & Trace Banner ████████████████████████████████████████████
108     intent = result.get("intent", "general")
109     trace = result.get("node_trace", [])
110 
111     col1, col2, col3 = st.columns(3)
112     col1.metric("Intent", intent)
113     col2.metric("Latency", f"{elapsed:.1f}s")
114     col3.metric("Nodes", " → ".join(trace))
115 
116     st.divider()
117 
118     # ■■ Final Answer (with inline Mermaid rendering) ████████████████████████
119     final_answer = result.get("final_answer", "")
120     mermaid = result.get("mermaid_diagram", "")
121 
122     # Build the stored content (diagram embedded so history replays it)
123     response_content = final_answer
124     if mermaid and f"`mermaid" not in final_answer:
125         # Synthesis didn't include the block inline – append it so the
126         # stored message and the expander both have the code available
127         response_content += f"\n\n`mermaid\n{mermaid}\n`"
128 
129     render_message(response_content)
130 
131     # ■■ Raw code expander (only when a diagram exists) ████████████████████████
132     if mermaid:
133         with st.expander("Raw Mermaid code (copy for mermaid.live)":
134             st.code(mermaid, language="text")
135 
136     # ■■ IRAC Analysis ████████████████████████████████████████████████████████████
137     irac = result.get("irac_analysis", "")
138     if irac:
139         with st.expander("Full IRAC Analysis", expanded=False):
140             st.markdown(irac)
141 
142     # ■■ Sources Panel ████████████████████████████████████████████████████████████
143     texts = result.get("retrieved_texts", [])
144     slides = result.get("retrieved_slides", [])
145     if texts or slides:
146         with st.expander(
147             f"Sources ({len(texts)} texts, {len(slides)} slides)", expanded=False
148         ):
149             if texts:
150                 st.markdown("***Text Sources:**")
151                 for t in texts:
152                     st.markdown(
153                         f"- [{t['week']}] **{t['source']}** "  

154                             f"[{t['doc_type']], {len(t['content'])} chars)"
155                     )
156             if slides:
157                 st.markdown("***Slide Sources:**")
158                 for s in slides:
159                     st.markdown(f"- [{s['week']}] **{s['source']}**")

```

```
161     st.session_state.messages.append({"role": "assistant", "content": response_content})
```

tests/test_phase1_indexing.py

Phase 1 tests: indexing

[illegible]

[illegible]

```
161     for meta in slides["metadatas"]:  
162         assert "image_path" in meta, f"Slide missing 'image_path': {meta}"  
163         assert meta["image_path"], "image_path is empty"
```

Phase 2 tests: retrieval and state

[illegible]

[illegible]

```

161         image_path=None,
162     )
163
164
165 class TestCrossEncoderReranker:
166     def test_empty_input_returns_empty(self) -> None:
167         from src.agent.reranker import CrossEncoderReranker
168
169         r = CrossEncoderReranker()
170         assert r.rerank("query", [], top_k=4) == []
171
172     def test_top_k_zero_returns_empty(self) -> None:
173         from src.agent.reranker import CrossEncoderReranker
174
175         r = CrossEncoderReranker()
176         docs = [_doc("a"), _doc("b")]
177         assert r.rerank("q", docs, top_k=0) == []
178
179     def test_top_k_truncates_and_orders_by_score(self) -> None:
180         """Higher cross-encoder score → earlier in returned list; truncated to top_k."""
181         from src.agent.reranker import CrossEncoderReranker
182
183         r = CrossEncoderReranker()
184
185         class FakeModel:
186             def predict(self, pairs):
187                 # Score each doc by length of its content (deterministic mock)
188                 return [float(len(p[1])) for p in pairs]
189
190         r._model = FakeModel()
191
192         docs = [
193             _doc("aa", source="short.pdf"),
194             _doc("aaaaa", source="medium.pdf"),
195             _doc("aaaaaaaaa", source="long.pdf"),
196             _doc("a", source="tiny.pdf"),
197         ]
198
199         ranked = r.rerank("query", docs, top_k=2)
200         assert len(ranked) == 2
201         assert ranked[0]["source"] == "long.pdf"
202         assert ranked[1]["source"] == "medium.pdf"
203
204     def test_reranker_falls_back_to_input_order_on_failure(self) -> None:
205         """If predict raises, return docs[:top_k] without crashing."""
206         from src.agent.reranker import CrossEncoderReranker
207
208         r = CrossEncoderReranker()
209
210         class BrokenModel:
211             def predict(self, pairs):
212                 raise RuntimeError("model exploded")
213
214         r._model = BrokenModel()
215
216         docs = [_doc("a", source="first.pdf"), _doc("b", source="second.pdf")]
217         ranked = r.rerank("q", docs, top_k=1)
218         assert len(ranked) == 1
219         assert ranked[0]["source"] == "first.pdf"
220
221     def test_singleton_get_reranker_returns_same_instance(self) -> None:
222         from src.agent import reranker as rer
223
224         rer._singleton = None
225         a = rer.get_reranker()
226         b = rer.get_reranker()
227         assert a is b
228
229
230 class TestRetrieveWithRerankerHook:
231     def test_use_reranker_false_skips_reranker(self) -> None:
232         """Passing use_reranker=False must call MMR with k=k and skip rerank entirely."""
233         from src.agent import tools
234
235         recorded: dict[str, int] = {}
236
237         class FakeStore:
238             def max_marginal_relevance_search(self, query, k, fetch_k, lambda_mult, filter):
239                 recorded["k"] = k
240                 return []
241
242             def similarity_search(self, query, k, filter):
243                 recorded["k"] = k

```

```

244         return []
245
246     with patch.object(tools, "_get_vectorstore", return_value=FakeStore()):
247         with patch.object(tools, "_maybe_rerank", side_effect=lambda q, d, top_k, use_reranker: d) as mr:
248             tools.retrieve_texts("q", k=8, use_reranker=False)
249             # k passed to MMR should equal user-requested k (no over-fetch)
250             assert recorded["k"] == 8
251             # _maybe_rerank still called but with use_reranker=False (no-op path)
252             mr.assert_called_once()
253             assert mr.call_args.kwargs["use_reranker"] is False
254
255     def test_use_reranker_true_overfetches_then_reranks(self) -> None:
256         """Passing use_reranker=True must over-fetch RERANKER_FETCH_K_TEXT candidates."""
257         from src.agent import tools
258         from src.config import RERANKER_FETCH_K_TEXT
259
260         recorded: dict[str, int] = {}
261
262         class FakeStore:
263             def max_marginal_relevance_search(self, query, k, fetch_k, lambda_mult, filter):
264                 recorded["k"] = k
265                 return []
266
267             def similarity_search(self, query, k, filter):
268                 recorded["k"] = k
269                 return []
270
271         with patch.object(tools, "_get_vectorstore", return_value=FakeStore()):
272             with patch.object(tools, "_maybe_rerank", side_effect=lambda q, d, top_k, use_reranker: d[:top_k])
273             as mr:
274                 tools.retrieve_texts("q", k=8, use_reranker=True)
275                 assert recorded["k"] == RERANKER_FETCH_K_TEXT
276                 mr.assert_called_once()
277                 assert mr.call_args.kwargs["use_reranker"] is True
278                 assert mr.call_args.kwargs["top_k"] == 8
279
280     @pytest.mark.integration
281     @pytest.mark.slow
282     class TestRerankerIntegration:
283         def test_real_reranker_against_three_chunks(self) -> None:
284             """Sanity-check the real cross-encoder against three hand-built chunks."""
285             from src.agent.reranker import CrossEncoderReranker
286
287             chunks = [
288                 _doc(
289                     "Adverse possession requires factual possession plus the intention "
290                     "to possess (animus possidendi) for the limitation period.",
291                     source="ap.pdf",
292                 ),
293                 _doc(
294                     "A fee simple is the largest estate known to law and is freely "
295                     "transferable inter vivos and on death.",
296                     source="estates.pdf",
297                 ),
298                 _doc(
299                     "The Torrens system replaced deeds-based registration with a "
300                     "central register conferring indefeasible title.",
301                     source="torrens.pdf",
302                 ),
303             ]
304             r = CrossEncoderReranker()
305             ranked = r.rerank("What are the elements of adverse possession?", chunks, top_k=3)
306
307             assert len(ranked) == 3
308             assert ranked[0]["source"] == "ap.pdf"

```

tests/test_phase3_nodes.py

Phase 3 tests: nodes

[illegible]

[illegible]

```

161
162 class TestRatioExtractorNode:
163     @pytest.mark.integration
164     @pytest.mark.slow
165     def test_produces_irac_and_ratio(self) -> None:
166         from src.agent.nodes import ratio_extractor_node, retrieval_node
167
168         state: AgentState = {
169             "query": "What is the ratio decidendi for adverse possession?",
170         }
171         state.update(retrieval_node(state))
172         result = ratio_extractor_node(state)
173
174         assert result["ratio_decidendi"], "ratio_decidendi should not be empty"
175         assert result["irac_analysis"], "irac_analysis should not be empty"
176         assert "ratio_extractor" in result["node_trace"]
177
178
179 class TestChronologyNode:
180     @pytest.mark.integration
181     @pytest.mark.slow
182     def test_produces_mermaid_diagram(self) -> None:
183         from src.agent.nodes import chronology_node, retrieval_node
184
185         state: AgentState = {
186             "query": "Show the timeline for adverse possession from week 3",
187             "week_filter": "week_3",
188         }
189         state.update(retrieval_node(state))
190         result = chronology_node(state)
191
192         assert result["mermaid_diagram"], "mermaid_diagram should not be empty"
193         assert "graph" in result["mermaid_diagram"].lower()
194         assert "chronology" in result["node_trace"]
195
196
197 class TestSynthesisNode:
198     @pytest.mark.integration
199     @pytest.mark.slow
200     def test_produces_final_answer(self) -> None:
201         from src.agent.nodes import retrieval_node, synthesis_node
202
203         state: AgentState = {
204             "query": "Explain the concept of fee simple",
205             "intent": "summary",
206         }
207         state.update(retrieval_node(state))
208         result = synthesis_node(state)
209
210         assert result["final_answer"], "final_answer should not be empty"
211         assert len(result["final_answer"]) > 50
212         assert "synthesis" in result["node_trace"]

```

tests/test_phase4_graph.py

Phase 4 tests: graph

[illegible]

[illegible]

[illegible]

```
244         from src.agent.verification import extract_case_citations
245
246         cites = extract_case_citations("In Smith vs Jones the rule was applied.")
247         assert "Smith vs Jones" in cites or "Smith v Jones" in cites
248
249     def test_dedup_case_insensitive(self) -> None:
250         from src.agent.verification import extract_case_citations
251
252         cites = extract_case_citations("Perry v Clissold ... again Perry v Clissold.")
253         assert len(cites) == 1
254
255     def test_ignores_lowercase_left(self) -> None:
256         from src.agent.verification import extract_case_citations
257
258         # "the v Other" should not match – lowercased determiner on the left.
259         cites = extract_case_citations("on the v Other side of things")
260         assert cites == []
```

tests/test_phase5_eval.py

Phase 5 tests: evaluation

```
1  """Phase 5 tests: evaluation framework, baselines, and plotting."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6
7  import pytest
8
9
10 def _mock_agent(**kwargs) -> dict:
11     defaults = {
12         "groundedness": {"score": 5},
13         "answer_relevancy": {"score": 5},
14         "latency_s": 10.0,
15         "source_diversity": 3,
16         "node_latencies": {"router": 2.0, "retrieval": 1.0, "synthesis": 7.0},
17         "mermaid_validity": {"score": 0.0},
18         "irac_compliance": {"score": 0.0},
19         "context_precision": {
20             "precision_at_k": 0.5,
21             "mrr": 0.5,
22             "hit_at_k": 1.0,
23             "ndcg_at_k": 0.8,
24             "k": 12,
25         },
26         "token_usage": {"total_input_tokens": 100, "total_output_tokens": 50, "total_tokens": 150},
27     }
28     defaults.update(kwargs)
29     return defaults
30
31
32 def _mock_baseline(**kwargs) -> dict:
33     defaults = {
34         "groundedness": {"score": 3},
35         "answer_relevancy": {"score": 4},
36         "latency_s": 5.0,
37         "source_diversity": 0,
38         "mermaid_validity": {"score": 0.0},
39         "irac_compliance": {"score": 0.0},
40         "token_usage": {"total_input_tokens": 10, "total_output_tokens": 20, "total_tokens": 30},
41     }
42     defaults.update(kwargs)
43     return defaults
44
45
46 def _mock_ablation(**kwargs) -> dict:
47     defaults = {
48         "groundedness": {"score": 4},
49         "answer_relevancy": {"score": 5},
50         "latency_s": 7.0,
51         "source_diversity": 3,
52         "mermaid_validity": {"score": 0.0},
53         "irac_compliance": {"score": 0.0},
54         "context_precision": {
55             "precision_at_k": 0.5,
56             "mrr": 0.5,
57             "hit_at_k": 1.0,
58             "ndcg_at_k": 0.8,
59             "k": 12,
60         },
61         "token_usage": {"total_input_tokens": 80, "total_output_tokens": 40, "total_tokens": 120},
62     }
63     defaults.update(kwargs)
64     return defaults
65
66
67 class TestRetrievalRankingMetrics:
68     def test_mrr_first_relevant_at_two(self) -> None:
69         from src.eval.run_evals import _ranking_metrics_from_binary_verdicts
70
71         m = _ranking_metrics_from_binary_verdicts([False, True, False])
72         assert m["mrr"] == 0.5
73         assert m["hit_at_k"] == 1.0
74         assert m["k"] == 3
75
76     def test_all_irrelevant(self) -> None:
77         from src.eval.run_evals import _ranking_metrics_from_binary_verdicts
```

```

78
79     m = _ranking_metrics_from_binary_verdicts([False, False])
80     assert m["mrr"] == 0.0
81     assert m["hit_at_k"] == 0.0
82
83
84 class TestEvalCaseHelpers:
85     def test_judge_question_multi_turn_includes_prior_turns(self) -> None:
86         from src.eval.run_evals import EVAL_CASES, _eval_case_last_turn, _judge_question_for_case
87
88         conv = next(c for c in EVAL_CASES if c["case_id"] == "conv_ap_followup")
89         assert _eval_case_last_turn(conv) == conv["user_turns"][-1]
90         jq = _judge_question_for_case(conv)
91         assert "Multi-turn" in jq
92         assert conv["user_turns"][0] in jq
93         assert conv["user_turns"][-1] in jq
94
95
96 class TestEvalImports:
97     def test_eval_module_imports(self) -> None:
98         from src.eval.run_evals import (
99             EVAL_CASES,
100             TEST_QUERIES,
101             judge_groundedness,
102             run_agent_with_metrics,
103             run_baseline,
104             run_evaluation,
105         )
106
107         assert len(EVAL_CASES) >= 8
108         assert len(TEST_QUERIES) == len(EVAL_CASES)
109
110     def test_eval_cases_cover_all_query_families(self) -> None:
111         from src.eval.run_evals import EVAL_CASES
112
113         fams = {c["query_family"] for c in EVAL_CASES}
114         assert fams == {
115             "factual_retrieval",
116             "cross_modal_retrieval",
117             "analytical_synthesis",
118             "conversational_followup",
119         }
120
121     def test_plot_generation_imports(self) -> None:
122         from src.eval.run_evals import _compute_summary, _generate_plots
123
124         assert callable(_compute_summary)
125         assert callable(_generate_plots)
126
127
128 class TestComputeSummary:
129     def test_summary_from_mock_results(self) -> None:
130         from src.eval.run_evals import _compute_summary
131
132         mock_results = [
133             {
134                 "query": "test",
135                 "query_family": "factual_retrieval",
136                 "case_id": "mock_a",
137                 "agent": _mock_agent(),
138                 "baseline": _mock_baseline(),
139                 "ablation_no_ratio": _mock_ablation(),
140             }
141         ]
142
143         summary = _compute_summary(mock_results)
144         assert summary["agent"]["avg_groundedness"] == 5.0
145         assert summary["baseline"]["avg_groundedness"] == 3.0
146         assert summary["agent"]["avg_latency_s"] == 10.0
147         assert summary["baseline"]["avg_source_diversity"] == 0.0
148         assert "avg_node_latencies" in summary["agent"]
149         assert summary["agent"]["avg_node_latencies"]["router"] == 2.0
150         assert "by_query_family" in summary
151         assert "factual_retrieval" in summary["by_query_family"]
152
153     def test_summary_handles_multiple_results(self) -> None:
154         from src.eval.run_evals import _compute_summary
155
156         mock_results = [
157             {
158                 "query": "q1",
159                 "query_family": "factual_retrieval",
160                 "case_id": "m1",

```

```

161         "agent": _mock_agent(groundedness={"score": 4}, node_latencies={"router": 2.0}),
162         "baseline": _mock_baseline(groundedness={"score": 2}),
163         "ablation_no_ratio": _mock_ablation(groundedness={"score": 3}),
164     },
165     {
166         "query": "q2",
167         "query_family": "analytical_synthesis",
168         "case_id": "m2",
169         "agent": _mock_agent(groundedness={"score": 5}, node_latencies={"router": 4.0}),
170         "baseline": _mock_baseline(groundedness={"score": 4}),
171         "ablation_no_ratio": _mock_ablation(groundedness={"score": 5}),
172     },
173 ]
174 summary = _compute_summary(mock_results)
175 assert summary["agent"]["avg_groundedness"] == 4.5
176 assert summary["agent"]["avg_node_latencies"]["router"] == 3.0
177
178
179 class TestPlotGeneration:
180     def test_plots_generate_without_error(self, tmp_path: Path) -> None:
181         from src.eval.run_evals import _compute_summary, _generate_plots
182
183         mock_results = [
184             {
185                 "query": "What is adverse possession?",
186                 "query_family": "factual_retrieval",
187                 "case_id": "p1",
188                 "agent": _mock_agent(
189                     groundedness={"score": 5},
190                     latency_s=30.0,
191                     source_diversity=3,
192                     node_latencies={"router": 3.0, "retrieval": 2.0, "ratio_extractor": 15.0, "synthesis": 10.
0},
193                 ),
194                 "baseline": _mock_baseline(groundedness={"score": 3}, latency_s=8.0),
195                 "ablation_no_ratio": _mock_ablation(groundedness={"score": 4}, latency_s=15.0),
196             },
197             {
198                 "query": "Explain the Torrens system",
199                 "query_family": "analytical_synthesis",
200                 "case_id": "p2",
201                 "agent": _mock_agent(
202                     groundedness={"score": 4},
203                     latency_s=25.0,
204                     source_diversity=2,
205                     node_latencies={"router": 2.5, "retrieval": 1.5, "synthesis": 12.0},
206                 ),
207                 "baseline": _mock_baseline(groundedness={"score": 2}, latency_s=6.0),
208                 "ablation_no_ratio": _mock_ablation(groundedness={"score": 3}, latency_s=12.0),
209             },
210         ]
211
212         summary = _compute_summary(mock_results)
213         _generate_plots(mock_results, summary, tmp_path)
214
215         assert (tmp_path / "groundedness_comparison.png").exists()
216         assert (tmp_path / "latency_comparison.png").exists()
217         assert (tmp_path / "per_query_groundedness.png").exists()
218         assert (tmp_path / "source_diversity.png").exists()
219         assert (tmp_path / "node_latency_breakdown.png").exists()
220         assert (tmp_path / "groundedness_by_query_family.png").exists()
221         assert (tmp_path / "retrieval_ranking_metrics.png").exists()
222
223
224 class TestBaseline:
225     @pytest.mark.integration
226     def test_baseline_returns_answer(self) -> None:
227         from src.eval.run_evals import run_baseline
228
229         result = run_baseline("What is a fee simple estate?")
230         assert result["answer"], "Baseline should produce an answer"
231         assert result["latency_s"] > 0
232         assert result["source_diversity"] == 0
233         assert result["node_trace"] == ["baseline_llm"]
234
235
236 class TestAgentWithMetrics:
237     @pytest.mark.integration
238     @pytest.mark.slow
239     def test_agent_returns_node_latencies(self) -> None:
240         from src.eval.run_evals import run_agent_with_metrics
241
242         result = run_agent_with_metrics("What is adverse possession?", week="week_3")

```

```

243         assert "node_latencies" in result
244         assert "router" in result["node_latencies"]
245         assert "retrieval" in result["node_latencies"]
246         assert "synthesis" in result["node_latencies"]
247         assert result["context"], "Should capture context for judge"
248
249
250     class TestGroundednessJudge:
251         @pytest.mark.integration
252         def test_judge_returns_valid_score(self) -> None:
253             from src.eval.run_evals import judge_groundedness
254
255             result = judge_groundedness(
256                 query="What is adverse possession?",
257                 context="Adverse possession is a legal doctrine where a squatter gains title after 12 years.",
258                 answer="Adverse possession allows someone to gain title to land after occupying it for 12 years.",
259             )
260             assert 1 <= result["score"] <= 5
261             assert "reasoning" in result

```

tests/test_chat_memory.py

Chat memory tests

```
1  """Unit tests for multi-turn chat memory helpers (no API calls)."""
2
3  from __future__ import annotations
4
5  from src.agent.chat_memory import (
6      build_retrieval_query,
7      format_transcript_for_llm,
8      get_chat_history,
9      prepare_chat_history_for_run,
10 )
11
12
13 def test_prepare_drops_invalid_and_truncates_count() -> None:
14     raw = [
15         {"role": "user", "content": "Hi"},
16         {"role": "assistant", "content": "Hello"},
17         {"role": "bogus", "content": "x"},
18         {"role": "user", "content": ""},
19     ]
20     out = prepare_chat_history_for_run(raw)
21     assert len(out) == 2
22     assert out[0]["role"] == "user"
23     assert out[1]["role"] == "assistant"
24
25
26 def test_format_transcript_numbered_labels() -> None:
27     msgs = prepare_chat_history_for_run(
28         [
29             {"role": "user", "content": "What is AP?"},
30             {"role": "assistant", "content": "Adverse possession is ..."},
31         ]
32     )
33     text = format_transcript_for_llm(msgs)
34     assert "[1] Student:" in text
35     assert "[2] Tutor:" in text
36     assert "Adverse possession" in text
37
38
39 def test_retrieval_query_standalone_is_unchanged() -> None:
40     assert build_retrieval_query("fee simple", []) == "fee simple"
41
42
43 def test_retrieval_query_includes_prior_answer_excerpt() -> None:
44     hist = prepare_chat_history_for_run(
45         [
46             {"role": "user", "content": "Explain bailment"},
47             {"role": "assistant", "content": "Bailment is delivery of goods ..." * 50},
48         ]
49     )
50     q = build_retrieval_query("Give one exam tip", hist)
51     assert "New student question:" in q
52     assert "Give one exam tip" in q
53     assert "Prior tutor answer" in q
54     assert "Bailment" in q
55
56
57 def test_get_chat_history_safe() -> None:
58     assert get_chat_history({"query": "x"}) == []
59     assert get_chat_history({"query": "x", "chat_history": "bad"}) == []
```

eval_results/eval_summary.json

Aggregate eval metrics

```
1  {
2    "agent": {
3      "avg_groundedness": 4.86,
4      "avg_answer_relevancy": 5.0,
5      "avg_mermaid_validity": 0.68,
6      "avg_irac_compliance": 1.0,
7      "irac_applicable_count": 19,
8      "irac_not_applicable_count": 3,
9      "avg_latency_s": 23.57,
10     "avg_source_diversity": 5.91,
11     "groundedness_scores": [
12       4,
13       5,
14       5,
15       5,
16       5,
17       5,
18       5,
19       5,
20       5,
21       5,
22       5,
23       5,
24       5,
25       5,
26       5,
27       5,
28       4,
29       5,
30       5,
31       5,
32       4,
33       5
34     ],
35     "answer_relevancy_scores": [
36       5,
37       5,
38       5,
39       5,
40       5,
41       5,
42       5,
43       5,
44       5,
45       5,
46       5,
47       5,
48       5,
49       5,
50       5,
51       5,
52       5,
53       5,
54       5,
55       5,
56       5,
57       5
58     ],
59     "mermaid_scores": [
60       0.0,
61       1.0,
62       0.0,
63       0.0,
64       0.0,
65       0.0,
66       1.0,
67       1.0,
68       1.0,
69       1.0,
70       1.0,
71       1.0,
72       0.0,
73       1.0,
74       1.0,
75       1.0,
76       1.0,
77       1.0,
```

```

78         1.0,
79         1.0,
80         1.0,
81         0.0
82     ],
83     "irac_scores": [
84         1.0,
85         1.0,
86         1.0,
87         1.0,
88         1.0,
89         1.0,
90         null,
91         1.0,
92         1.0,
93         null,
94         1.0,
95         1.0,
96         1.0,
97         1.0,
98         1.0,
99         1.0,
100        1.0,
101        1.0,
102        1.0,
103        1.0,
104        null,
105        1.0
106    ],
107    "latencies": [
108        24.1,
109        24.88,
110        12.01,
111        12.02,
112        10.38,
113        15.0,
114        16.85,
115        27.35,
116        26.64,
117        19.56,
118        30.66,
119        27.85,
120        12.92,
121        21.4,
122        31.76,
123        26.72,
124        26.52,
125        36.67,
126        34.82,
127        33.85,
128        31.44,
129        15.05
130    ],
131    "avg_context_precision": 0.66,
132    "context_precisions": [
133        0.17,
134        0.67,
135        0.67,
136        0.67,
137        0.58,
138        0.67,
139        0.83,
140        0.92,
141        0.42,
142        0.83,
143        0.67,
144        0.92,
145        1.0,
146        0.67,
147        0.83,
148        0.58,
149        0.33,
150        0.67,
151        0.58,
152        0.92,
153        0.5,
154        0.5
155    ],
156    "avg_mrr": 0.97,
157    "avg_hit_at_k": 1.0,
158    "avg_ndcg_at_k": 0.94,
159    "mrr_per_query": [
160        1.0,

```

```

161     1.0,
162     1.0,
163     1.0,
164     1.0,
165     1.0,
166     1.0,
167     1.0,
168     1.0,
169     1.0,
170     1.0,
171     1.0,
172     1.0,
173     1.0,
174     1.0,
175     1.0,
176     1.0,
177     1.0,
178     1.0,
179     1.0,
180     1.0,
181     0.333
182 ],
183 "hit_at_k_per_query": [
184     1.0,
185     1.0,
186     1.0,
187     1.0,
188     1.0,
189     1.0,
190     1.0,
191     1.0,
192     1.0,
193     1.0,
194     1.0,
195     1.0,
196     1.0,
197     1.0,
198     1.0,
199     1.0,
200     1.0,
201     1.0,
202     1.0,
203     1.0,
204     1.0,
205     1.0
206 ],
207 "ndcg_at_k_per_query": [
208     0.79,
209     1.0,
210     0.9,
211     0.978,
212     0.926,
213     0.992,
214     0.991,
215     0.996,
216     0.826,
217     0.995,
218     1.0,
219     1.0,
220     1.0,
221     0.98,
222     1.0,
223     0.989,
224     0.809,
225     1.0,
226     0.949,
227     0.998,
228     0.905,
229     0.654
230 ],
231 "total_input_tokens": 541915,
232 "total_output_tokens": 33257,
233 "total_tokens": 575172,
234 "avg_tokens_per_query": 26144,
235 "avg_node_latencies": {
236     "chronology": 8.6,
237     "ratio_extractor": 6.56,
238     "retrieval": 2.73,
239     "router": 1.08,
240     "synthesis": 4.59,
241     "verification": 0.0
242 }
243 },

```

```

244 "baseline": {
245     "avg_groundedness": 3.41,
246     "avg_answer_relevancy": 4.5,
247     "avg_mermaid_validity": 0.0,
248     "avg_irac_compliance": 0.1,
249     "irac_applicable_count": 22,
250     "irac_not_applicable_count": 0,
251     "avg_latency_s": 3.72,
252     "avg_source_diversity": 0.0,
253     "groundedness_scores": [
254         1,
255         4,
256         4,
257         5,
258         5,
259         2,
260         4,
261         4,
262         3,
263         4,
264         4,
265         3,
266         2,
267         5,
268         4,
269         5,
270         4,
271         4,
272         1,
273         4,
274         2,
275         1
276     ],
277     "answer_relevancy_scores": [
278         1,
279         5,
280         5,
281         5,
282         5,
283         5,
284         5,
285         5,
286         5,
287         5,
288         5,
289         5,
290         3,
291         5,
292         5,
293         5,
294         4,
295         5,
296         2,
297         5,
298         4,
299         5
300     ],
301     "mermaid_scores": [
302         0.0,
303         0.0,
304         0.0,
305         0.0,
306         0.0,
307         0.0,
308         0.0,
309         0.0,
310         0.0,
311         0.0,
312         0.0,
313         0.0,
314         0.0,
315         0.0,
316         0.0,
317         0.0,
318         0.0,
319         0.0,
320         0.0,
321         0.0,
322         0.0,
323         0.0
324     ],
325     "irac_scores": [
326         0.25,

```

```

327         0.0,
328         0.25,
329         0.0,
330         0.0,
331         0.25,
332         0.0,
333         0.0,
334         0.25,
335         0.0,
336         0.25,
337         0.5,
338         0.0,
339         0.0,
340         0.25,
341         0.0,
342         0.0,
343         0.0,
344         0.0,
345         0.25,
346         0.0,
347         0.0
348     ],
349     "latencies": [
350         3.5,
351         4.81,
352         2.52,
353         2.42,
354         1.82,
355         2.21,
356         3.64,
357         9.31,
358         2.64,
359         4.3,
360         6.0,
361         6.55,
362         1.57,
363         2.05,
364         3.05,
365         3.07,
366         2.77,
367         3.42,
368         4.05,
369         5.84,
370         3.99,
371         2.25
372     ],
373     "total_input_tokens": 1049,
374     "total_output_tokens": 6815,
375     "total_tokens": 7864,
376     "avg_tokens_per_query": 357
377 },
378 "ablation_mega_prompt": {
379     "avg_groundedness": 4.77,
380     "avg_answer_relevancy": 4.95,
381     "avg_mermaid_validity": 0.59,
382     "avg_irac_compliance": 1.0,
383     "irac_applicable_count": 22,
384     "irac_not_applicable_count": 0,
385     "avg_latency_s": 9.52,
386     "avg_source_diversity": 6.05,
387     "groundedness_scores": [
388         4,
389         5,
390         5,
391         5,
392         5,
393         5,
394         5,
395         5,
396         5,
397         5,
398         5,
399         5,
400         5,
401         4,
402         5,
403         5,
404         4,
405         5,
406         5,
407         5,
408         4,
409         4

```

```
410 ],
411 "answer_relevancy_scores": [
412     5,
413     5,
414     5,
415     5,
416     5,
417     5,
418     5,
419     5,
420     5,
421     5,
422     5,
423     5,
424     5,
425     4,
426     5,
427     5,
428     5,
429     5,
430     5,
431     5,
432     5,
433     5
434 ],
435 "mermaid_scores": [
436     0.0,
437     0.0,
438     1.0,
439     1.0,
440     1.0,
441     1.0,
442     1.0,
443     1.0,
444     0.0,
445     1.0,
446     0.0,
447     1.0,
448     0.0,
449     0.0,
450     1.0,
451     0.0,
452     0.0,
453     1.0,
454     1.0,
455     1.0,
456     1.0,
457     0.0
458 ],
459 "irac_scores": [
460     1.0,
461     1.0,
462     1.0,
463     1.0,
464     1.0,
465     1.0,
466     1.0,
467     1.0,
468     1.0,
469     1.0,
470     1.0,
471     1.0,
472     1.0,
473     1.0,
474     1.0,
475     1.0,
476     1.0,
477     1.0,
478     1.0,
479     1.0,
480     1.0,
481     1.0
482 ],
483 "latencies": [
484     10.11,
485     7.27,
486     9.83,
487     8.64,
488     7.15,
489     11.03,
490     9.81,
491     10.44,
492     7.99,
```

```

493     9.65,
494     11.47,
495     11.37,
496     6.41,
497     7.38,
498     11.98,
499     8.09,
500     9.93,
501     10.08,
502     9.78,
503     11.37,
504     8.32,
505     11.3
506 ],
507 "avg_context_precision": 0.65,
508 "context_precisions": [
509     0.17,
510     0.67,
511     0.58,
512     0.67,
513     0.5,
514     0.67,
515     0.83,
516     0.92,
517     0.5,
518     0.75,
519     0.67,
520     0.92,
521     1.0,
522     0.58,
523     0.92,
524     0.58,
525     0.25,
526     0.67,
527     0.75,
528     0.83,
529     0.5,
530     0.33
531 ],
532 "avg_mrr": 0.97,
533 "avg_hit_at_k": 1.0,
534 "avg_ndcg_at_k": 0.94,
535 "mrr_per_query": [
536     1.0,
537     1.0,
538     1.0,
539     1.0,
540     1.0,
541     1.0,
542     1.0,
543     1.0,
544     1.0,
545     1.0,
546     1.0,
547     1.0,
548     1.0,
549     1.0,
550     1.0,
551     1.0,
552     1.0,
553     1.0,
554     1.0,
555     1.0,
556     1.0,
557     0.333
558 ],
559 "hit_at_k_per_query": [
560     1.0,
561     1.0,
562     1.0,
563     1.0,
564     1.0,
565     1.0,
566     1.0,
567     1.0,
568     1.0,
569     1.0,
570     1.0,
571     1.0,
572     1.0,
573     1.0,
574     1.0,
575     1.0,

```

```

576         1.0,
577         1.0,
578         1.0,
579         1.0,
580         1.0,
581         1.0
582     ],
583     "ndcg_at_k_per_query": [
584         0.79,
585         1.0,
586         0.891,
587         0.978,
588         0.829,
589         0.992,
590         0.991,
591         0.996,
592         0.928,
593         0.997,
594         1.0,
595         1.0,
596         1.0,
597         0.982,
598         0.994,
599         0.989,
600         0.845,
601         1.0,
602         0.991,
603         1.0,
604         0.905,
605         0.591
606     ],
607     "total_input_tokens": 201294,
608     "total_output_tokens": 25751,
609     "total_tokens": 227045,
610     "avg_tokens_per_query": 10320
611 },
612 "ablation_no_reranker": {
613     "avg_groundedness": 4.86,
614     "avg_answer_relevancy": 5.0,
615     "avg_mermaid_validity": 0.68,
616     "avg_irac_compliance": 1.0,
617     "irac_applicable_count": 18,
618     "irac_not_applicable_count": 4,
619     "avg_latency_s": 22.11,
620     "avg_source_diversity": 6.05,
621     "groundedness_scores": [
622         5,
623         5,
624         5,
625         5,
626         5,
627         5,
628         5,
629         5,
630         5,
631         5,
632         5,
633         5,
634         4,
635         5,
636         5,
637         5,
638         5,
639         5,
640         5,
641         4,
642         4,
643         5
644     ],
645     "answer_relevancy_scores": [
646         5,
647         5,
648         5,
649         5,
650         5,
651         5,
652         5,
653         5,
654         5,
655         5,
656         5,
657         5,
658         5,

```

```

659         5,
660         5,
661         5,
662         5,
663         5,
664         5,
665         5,
666         5,
667         5
668     ],
669     "mermaid_scores": [
670         0.0,
671         1.0,
672         0.0,
673         0.0,
674         0.0,
675         0.0,
676         1.0,
677         1.0,
678         1.0,
679         1.0,
680         1.0,
681         1.0,
682         0.0,
683         1.0,
684         1.0,
685         1.0,
686         1.0,
687         1.0,
688         1.0,
689         1.0,
690         1.0,
691         0.0
692     ],
693     "irac_scores": [
694         1.0,
695         1.0,
696         1.0,
697         1.0,
698         1.0,
699         1.0,
700         null,
701         1.0,
702         1.0,
703         null,
704         1.0,
705         1.0,
706         null,
707         1.0,
708         1.0,
709         1.0,
710         1.0,
711         1.0,
712         1.0,
713         1.0,
714         null,
715         1.0
716     ],
717     "latencies": [
718         12.5,
719         24.88,
720         12.75,
721         11.47,
722         10.44,
723         12.91,
724         20.13,
725         36.78,
726         26.21,
727         17.18,
728         28.77,
729         32.16,
730         6.55,
731         22.94,
732         35.33,
733         25.67,
734         28.75,
735         30.38,
736         19.56,
737         33.9,
738         20.56,
739         16.59
740     ],
741     "avg_context_precision": 0.59,

```

```

742 "context_precisions": [
743     0.17,
744     0.67,
745     0.5,
746     0.58,
747     0.5,
748     0.42,
749     0.67,
750     0.75,
751     0.42,
752     0.67,
753     0.67,
754     1.0,
755     0.83,
756     0.67,
757     0.83,
758     0.58,
759     0.08,
760     0.67,
761     0.5,
762     0.75,
763     0.58,
764     0.58
765 ],
766 "avg_mrr": 0.932,
767 "avg_hit_at_k": 1.0,
768 "avg_ndcg_at_k": 0.941,
769 "mrr_per_query": [
770     1.0,
771     1.0,
772     1.0,
773     1.0,
774     0.5,
775     1.0,
776     1.0,
777     1.0,
778     1.0,
779     1.0,
780     1.0,
781     1.0,
782     1.0,
783     1.0,
784     1.0,
785     1.0,
786     0.5,
787     1.0,
788     1.0,
789     1.0,
790     0.5,
791     1.0
792 ],
793 "hit_at_k_per_query": [
794     1.0,
795     1.0,
796     1.0,
797     1.0,
798     1.0,
799     1.0,
800     1.0,
801     1.0,
802     1.0,
803     1.0,
804     1.0,
805     1.0,
806     1.0,
807     1.0,
808     1.0,
809     1.0,
810     1.0,
811     1.0,
812     1.0,
813     1.0,
814     1.0,
815     1.0
816 ],
817 "ndcg_at_k_per_query": [
818     0.79,
819     1.0,
820     0.983,
821     0.985,
822     0.752,
823     0.971,
824     1.0,

```

```

825     1.0,
826     0.933,
827     1.0,
828     1.0,
829     1.0,
830     1.0,
831     1.0,
832     0.995,
833     0.995,
834     0.631,
835     1.0,
836     0.898,
837     1.0,
838     0.812,
839     0.964
840 ],
841 "total_input_tokens": 563584,
842 "total_output_tokens": 34911,
843 "total_tokens": 598495,
844 "avg_tokens_per_query": 27204
845 },
846 "by_query_family": {
847     "factual_retrieval": {
848         "description": "Direct retrieval of stored knowledge: doctrine, tests, and definitions that should appear verbatim or paraphrased from a small number of chunks.",
849         "case_count": 6,
850         "case_ids": [
851             "fact_moran_test",
852             "fact_fee_simple",
853             "fact_bailment_ratio",
854             "fact_perry_possessory",
855             "fact_fee_tail_vs_simple",
856             "fact_pla_chattels"
857         ],
858         "agent_avg_groundedness": 4.67,
859         "agent_avg_answer_relevancy": 5.0,
860         "agent_avg_context_precision": 0.52
861     },
862     "cross_modal_retrieval": {
863         "description": "Requires the lecture-slide channel: questions are phrased to target VLM-indexed slide descriptions alongside PDF readings (multimodal KB).",
864         "case_count": 5,
865         "case_ids": [
866             "xmodal_chattels_slides",
867             "xmodal_moukataff_slide",
868             "xmodal_ap_slides",
869             "xmodal_torrens_indefeasibility",
870             "xmodal_bailment_categories"
871         ],
872         "agent_avg_groundedness": 5.0,
873         "agent_avg_answer_relevancy": 5.0,
874         "agent_avg_context_precision": 0.67,
875         "fraction_cases_with_text_and_slides_retrieved": 1.0
876     },
877     "analytical_synthesis": {
878         "description": "Multi-evidence answers: compare, relate, summarise across concepts, or reconstruct sequences using several retrieved sources.",
879         "case_count": 7,
880         "case_ids": [
881             "anal_ap_elements",
882             "anal_lease_licence",
883             "anal_chronology_ap",
884             "anal_torrens_framework",
885             "anal_taxonomy",
886             "anal_legal_vs_equitable_interests",
887             "anal_torrens_title_passing"
888         ],
889         "agent_avg_groundedness": 4.86,
890         "agent_avg_answer_relevancy": 5.0,
891         "agent_avg_context_precision": 0.74
892     },
893     "conversational_followup": {
894         "description": "Multi-turn: first user message establishes topic; second message is a follow-up resolved using chat_history (baseline sees only the last turn).",
895         "case_count": 4,
896         "case_ids": [
897             "conv_ap_followup",
898             "conv_torrens_followup",
899             "conv_chattels_found",
900             "conv_lease_three_turn"
901         ],
902         "agent_avg_groundedness": 5.0,
903         "agent_avg_answer_relevancy": 5.0,

```

```
904         "agent_avg_context_precision": 0.75
905     }
906 }
907 }
```

eval_results/failure_analysis.md

Failure analysis report

```
1 # Failure Analysis Report
2
3 Auto-generated from evaluation results. This report identifies queries where the system underperformed and cat
4 egorises the root causes.
5
6 ## 0. Query family coverage
7
8 Every eval case is assigned exactly one of four families. Use this table for success/failure discussion pe
9 r family.
10
11 | Family | Cases | Case IDs |
12 |-----|-----|-----|
13 | factual_retrieval | 6 | fact_moran_test, fact_fee_simple, fact_bailment_ratio, fact_perry_possessory, fa
14 ct_fee_tail_vs_simple, fact_pla_chattels |
15 | cross_modal_retrieval | 5 | xmodal_chattels_slides, xmodal_moukataff_slide, xmodal_ap_slides, xmodal_tor
16 rens_indefeasibility, xmodal_bailment_categories |
17 | analytical_synthesis | 7 | anal_ap_elements, anal_lease_licence, anal_chronology_ap, anal_torrens_framew
18 ork, anal_taxonomy, anal_legal_vs_equitable_interests, anal_torrens_title_passing |
19 | conversational_followup | 4 | conv_ap_followup, conv_torrens_followup, conv_chattels_found, conv_lease_t
20 hree_turn |
21
22 - factual_retrieval: Direct retrieval of stored knowledge: doctrine, tests, and definitions that should ap
23 pear verbatim or paraphrased from a small number of chunks.
24 - cross_modal_retrieval: Requires the lecture-slide channel: questions are phrased to target VLM-indexed s
25 lide descriptions alongside PDF readings (multimodal KB).
26 - analytical_synthesis: Multi-evidence answers: compare, relate, summarise across concepts, or reconstruct
27 sequences using several retrieved sources.
28 - conversational_followup: Multi-turn: first user message establishes topic; second message is a follow-up
29 resolved using chat_history (baseline sees only the last turn).
30
31 ### Family: `factual_retrieval`
32
33 - fact_moran_test (single turn): groundedness 4/5, relevancy 5/5
34 - Rationale: Named case test – should retrieve specific ratio/test from readings.
35 - fact_fee_simple (single turn): groundedness 5/5, relevancy 5/5
36 - Rationale: Core estate definition – single-concept factual answer.
37 - fact_bailment_ratio (single turn): groundedness 5/5, relevancy 5/5
38 - Rationale: Ratio-style question tied to course materials on bailment.
39 - fact_perry_possessory (single turn): groundedness 5/5, relevancy 5/5
40 - Rationale: Named case holding from indexed readings.
41 - fact_fee_tail_vs_simple (single turn): groundedness 5/5, relevancy 5/5
42 - Rationale: Comparative doctrine question – should retrieve estate-type readings.
43 - fact_pla_chattels (single turn): groundedness 4/5, relevancy 5/5
44 - Rationale: Statute lookup – Property Law Act 1974 (Qld) on chattels.
45
46 ### Family: `cross_modal_retrieval`
47
48 - xmodal_chattels_slides (single turn): groundedness 5/5, relevancy 5/5
49 - Rationale: Forces reliance on VLM-described lecture slides for chattels possession.
50 - Retrieval: text chunks=8, slide chunks=4, both_modalities=True
51 - xmodal_moukataff_slide (single turn): groundedness 5/5, relevancy 5/5
52 - Rationale: Slide-anchored case (Moukataff) should appear in slide descriptions.
53 - Retrieval: text chunks=8, slide chunks=4, both_modalities=True
54 - xmodal_ap_slides (single turn): groundedness 5/5, relevancy 5/5
55 - Rationale: Asks for emphasis from slide-indexed adverse possession teaching.
56 - Retrieval: text chunks=8, slide chunks=4, both_modalities=True
57 - xmodal_torrens_indefeasibility (single turn): groundedness 5/5, relevancy 5/5
58 - Rationale: Targets lecture slides on Torrens indefeasibility – VLM-described content.
59 - Retrieval: text chunks=8, slide chunks=4, both_modalities=True
60 - xmodal_bailment_categories (single turn): groundedness 5/5, relevancy 5/5
61 - Rationale: Slide-anchored taxonomy: bailment categories from lecture deck.
62 - Retrieval: text chunks=8, slide chunks=4, both_modalities=True
63
64 ### Family: `analytical_synthesis`
65
66 - anal_ap_elements (single turn): groundedness 5/5, relevancy 5/5
67 - Rationale: Must relate factual possession and animus across sources.
68 - anal_lease_licence (single turn): groundedness 5/5, relevancy 5/5
69 - Rationale: Compare two institutions from materials (distinction test).
70 - anal_chronology_ap (single turn): groundedness 5/5, relevancy 5/5
71 - Rationale: Chronological reconstruction across multiple factual beats.
72 - anal_torrens_framework (single turn): groundedness 5/5, relevancy 5/5
73 - Rationale: Framework summary spanning readings/slides on Torrens.
74 - anal_taxonomy (single turn): groundedness 5/5, relevancy 5/5
75 - Rationale: Taxonomy spans multiple categories (real vs personal property).
76 - anal_legal_vs_equitable_interests (single turn): groundedness 5/5, relevancy 5/5
77 - Rationale: Comparison across multiple chunks of equitable vs legal interests.
```

```

68 - **anal_torrens_title_passing** (single turn): groundedness **4/5**, relevancy **5/5**
69 - *Rationale:* Sequenced reasoning: passage of legal title in a Torrens transfer.
70
71 ### Family: `conversational_followup`
72
73 - **conv_ap_followup** (2 turn(s)): groundedness **5/5**, relevancy **5/5**
74 - *Rationale:* Second turn references 'that ratio' – needs session memory.
75 - **conv_torrens_followup** (2 turn(s)): groundedness **5/5**, relevancy **5/5**
76 - *Rationale:* Contrast question after Torrens explanation – coreference to prior answer.
77 - **conv_chattels_found** (2 turn(s)): groundedness **5/5**, relevancy **5/5**
78 - *Rationale:* Hypothetical extension after test – requires prior topic + retrieval.
79 - **conv_lease_three_turn** (3 turn(s)): groundedness **5/5**, relevancy **5/5**
80 - *Rationale:* Three-turn case where the third turn references the second.
81
82 ## 1. Low Groundedness (Agent < 4/5)
83
84 No queries scored below 4. All agent answers were well-grounded.
85
86 ## 2. Ablation Outperformed Agent
87
88 Cases where the single mega-prompt outperformed the full agent on groundedness, suggesting the multi-node pipe
89 line introduced unverifiable inferences on those queries.
90
91 The agent matched or outperformed the ablation on all queries.
92
93 ## 3. Low Context Precision (Retrieval < 70%)
94
95 Queries where fewer than 70% of retrieved chunks were judged relevant, indicating the retrieval missed the tar
96 get topic.
97
98 ### Q: "What is the legal test from Buckinghamshire County Council v Moran?"
99 - **Precision@K:** 17% (2/12 relevant)
100 - **Verdicts:** [True, False, False, False, False, False, False, False, False, True, False, False]
101
102 ### Q: "What is a fee simple estate?"
103 - **Precision@K:** 67% (8/12 relevant)
104 - **Verdicts:** [True, True, True, True, True, True, True, True, False, False, False, False]
105
106 ### Q: "What is the ratio decidendi regarding bailment at will?"
107 - **Precision@K:** 67% (8/12 relevant)
108 - **Verdicts:** [True, False, True, True, True, False, True, True, True, True, False, False]
109
110 ### Q: "What is the ratio in Perry v Clissold regarding possessory title?"
111 - **Precision@K:** 67% (8/12 relevant)
112 - **Verdicts:** [True, True, True, True, False, True, True, True, True, False, False, False]
113
114 ### Q: "According to the indexed lecture slide materials in the knowledge base, what two elements are required
115 to establish possession of chattels?"
116 - **Precision@K:** 58% (7/12 relevant)
117 - **Verdicts:** [True, True, True, False, False, True, True, False, False, False, True, True]
118
119 ### Q: "What does the indexed lecture content describe about Moukataff v BOAC and baggage or chattel possessio
120 n?"
121 - **Precision@K:** 67% (8/12 relevant)
122 - **Verdicts:** [True, True, True, True, True, True, True, False, True, True, False, False]
123
124 ### Q: "What legal principle governs the distinction between a lease and a licence, and how does each concept
125 apply in outline?"
126 - **Precision@K:** 42% (5/12 relevant)
127 - **Verdicts:** [True, False, False, True, False, True, True, True, False, False, False, False]
128
129 ### Q: "Summarise the legal framework for land registration under the Torrens system as presented in the index
130 ed course materials."
131 - **Precision@K:** 67% (8/12 relevant)
132 - **Verdicts:** [True, True, True, True, True, True, True, True, False, False, False, False]
133
134 ### Q: "In one paragraph, how does that differ from a deeds registry? Stay within our indexed course sources o
135 nly."
136 - **Precision@K:** 67% (8/12 relevant)
137 - **Verdicts:** [True, True, True, True, True, False, True, True, False, False, True, False]
138
139 ### Q: "What is the difference between a fee tail and a fee simple in our materials?"
140 - **Precision@K:** 58% (7/12 relevant)
141 - **Verdicts:** [True, True, True, True, True, False, True, True, False, False, False, False]
142
143 ### Q: "What does the Property Law Act 1974 (Qld) say about chattels in the readings?"
144 - **Precision@K:** 33% (4/12 relevant)
145 - **Verdicts:** [True, False, True, False, False, False, False, False, True, False, False, True]
146
147 ### Q: "Looking at the indexed lecture slides on the Torrens system, how do those materials present the doctri
148 ne of indefeasibility of title?"
149 - **Precision@K:** 67% (8/12 relevant)
150 - **Verdicts:** [True, True, True, True, True, True, True, True, False, False, False, False]

```

```

143
144 ### Q: "From the lecture slides indexed in the knowledge base, what categories of bailment do those materials
145 describe?"
146 - **Precision@K:** 58% (7/12 relevant)
147 - **Verdicts:** [True, True, True, False, True, True, False, False, True, False, True, False]
148
149 ### Q: "Sequence the steps by which legal title passes in a Torrens-system land transfer, citing the indexed c
150 ourse materials at each step."
151 - **Precision@K:** 50% (6/12 relevant)
152 - **Verdicts:** [True, False, True, True, True, True, False, True, False, False, False, False]
153
154 ### Q: "Which of those points you just made would change if the licence were irrevocable? Stay within our inde
155 xed sources only."
156 - **Precision@K:** 50% (6/12 relevant)
157 - **Verdicts:** [False, False, True, False, True, True, True, True, False, False, False, True]
158
159 ## 4. Low Answer Relevancy (Agent < 4/5)
160
161 All agent answers scored 4+ on relevancy.
162
163 ## 5. Baseline Outperformed Agent
164
165 The agent matched or outperformed the baseline on all queries.
166
167 ## 5b. Reranker Ablation (Agent vs no-reranker)
168
169 Compares context precision between the full agent (cross-encoder reranker on) and the same agent run with `USE
170 _RERANKER=False`. This isolates the contribution of the reranker.
171
172 - Reranker improved precision on **11/22** queries
173 - Reranker hurt precision on **3/22** queries
174 - Tie on **8/22**
175
176 ## 6. Summary
177
178 | Category | Count | Percentage |
179 |-----|-----|-----|
180 | Total queries | 22 | 100% |
181 | Low groundedness (<4) | 0 | 0% |
182 | Ablation outperformed agent | 0 | 0% |
183 | Low context precision (<70%) | 15 | 68% |
184 | Low answer relevancy (<4) | 0 | 0% |
185 | Baseline outperformed agent | 0 | 0% |

```

eval_results/human_spot_check.md

Human spot check log

```
1 # Human Spot-Check Log
2
3 _This log exists to support qualitative validation of the LLM-as-a-judge_
4 _pipeline. Five evaluation cases have been sampled deterministically_
5 _(seed=4205). Fill in Human Score (1-5) and Comments by hand,_
6 _then re-run the harness or compute disagreement (mean absolute error_
7 _between Human Score and `judge_groundedness.score`) yourself._
8
9 Disagreement convention:  $|\text{human} - \text{judge}| \geq 2$  indicates a substantive
10 rating gap that should be discussed in the report.
11
12 ---
13
14 ## Spot-check 1 – `fact_perry_possessory` (factual_retrieval)
15
16 Question: What is the ratio in Perry v Clissold regarding possessory title?
17
18 Agent answer (excerpt):
19 ```
20
21 Ratio in *Perry v Clissold* on possessory title
22
23 The ratio is:
24
25 > A person in peaceful possession of land, in the assumed character of owner and exercising the ordinary rights of ownership, has a perfectly good possessory title against all the world except the rightful owner. (Source: Readings Week 3; Supplementary Notes)
26
27 ## In plain English
28 This means that possession itself gives a legal title of its own. So even if someone does not have documentary title, they can still have a title that is enforceable against everyone except someone with a better right to possession. (Source: Lecture 3 Slide 1; Supplementary Notes)
29
30 ## Why that matters in *Perry v Clissold*
31 Clissold had taken possession of the land, enclosed it, received rents, and paid rates and taxes. The Privy Council held that he was entitled to compensation when the land was compulsorily acquired, because his possessory title was enough. (Source: Readings Week 3)
32
33 ## Short exam-style formulation
34 Possession confers a possessory title, and that title is good against all but the true owner. (Source: Supplementary Notes; Lecture 3 Slide 1)
35
36 If you want, I can also turn this into a 2-3 sentence exam answer
37
38 ...[truncated for readability]
39 ```
40
41 Judge groundedness: 5/5
42 Judge answer relevancy: 5/5
43
44 Human groundedness (1-5): 5
45 Human answer relevancy (1-5): 5
46
47 Comments (free text):
48
49 >
50
51 ---
52
53 ## Spot-check 2 – `xmodal_moukataff_slide` (cross_modal_retrieval)
54
55 Question: What does the indexed lecture content describe about Moukataff v BOAC and baggage or chattel possession?
56
57 Agent answer (excerpt):
58 ```
59
60 Moukataff v BOAC: baggage, chattels, and possession
61
62 The lecture content uses Moukataff v BOAC [1967] 1 Lloyd's Rep 396 to show that you can have possession of a container and its contents even if you do not know the exact contents, so long as you knowingly take charge of the general class of items involved. (Source: Lecture 3 Slide 6; Supplementary Notes 77, 37)
63
64 ## What the case was about
65 - A bank sent £20,000 in four packages to Kuwait by registered mail. (Source: Supplementary Notes 77; Reading Week 3)
66 - The packages were given to BOAC for carriage by air. (Source: Supplementary Notes 37; Reading Week 3)
```

```

67 - An employee/loader stole the money while the parcels were being handled by BOAC. (Source: Supplementary Note
s 77; Reading Week 3)
68
69 ### What the lecture says the case stands for
70 The indexed lecture content says the key point is "intention to control":
71 - You do not need knowledge of the precise contents of what you physically control. (Source: Lecture 3 Sli
de 6)
72 - Browne J said it was "almost absurd" to describe BOAC as involuntary bailees, because:
73   - BOAC knew the mail bags contained mail; and
74   - the red-labelled registered mail bags
75
76 ...[truncated for readability]
77 ``
78
79 Judge groundedness: 5/5
80 Judge answer relevancy: 5/5
81
82 Human groundedness (1-5): 5
83 Human answer relevancy (1-5): 5
84
85 Comments (free text):
86
87 >
88
89 ---
90
91 ## Spot-check 3 – `fact_fee_simple` (factual_retrieval)
92
93 Question: What is a fee simple estate?
94
95 Agent answer (excerpt):
96
97 ``
98 ## Fee simple estate
99
100 A fee simple estate is the most extensive freehold estate known to the common law. It is the closest t
hing to "full ownership" of land for practical purposes, although under the doctrine of tenure land is still n
ot treated as absolutely owned in the modern sense. (Source: Supplementary Notes - Greg 16.02.2026.pdf)
101
102 ### Key features
103 - It is of uncertain duration and can potentially last indefinitely.
104 - It is inheritable: it can pass to heirs.
105 - It is alienable: the holder can transfer it, and can also create lesser interests out of it, such as l
ife estates and leases.
106 - It is unrestricted: unlike fee tail, it is not limited to particular heirs. (Source: Supplementary Notes
- Greg 16.02.2026.pdf)
107
108 ### In simple terms
109 If someone owns land in fee simple, they have the strongest form of estate in land. They can usually use, sell
, or pass it on, and the estate can continue for a very long time unless it is brought to an end by some event
. (Source: Supplementary Notes - Greg 16.02.2026.pdf)
110
111 ### Types of fee simple
112 The notes also say there are three types:
113 1. Fee simple absolute – perpetual and not determinable by any
114
115 ...[truncated for readability]
116 ``
117
118 Judge groundedness: 5/5
119 Judge answer relevancy: 5/5
120
121 Human groundedness (1-5): 5
122 Human answer relevancy (1-5): 5
123
124 Comments (free text):
125
126 >
127
128 ---
129
130 ## Spot-check 4 – `conv_ap_followup` (conversational_followup)
131
132 Question: Give one short exam tip tailored to that ratio for an open-book exam.
133
134 Agent answer (excerpt):
135
136 ``
137 One short open-book exam tip: use the facts in a checklist – first spot factual possession + intention t
o possess, then confirm it was open, peaceful, and without consent, and only after that move to the li
mitation period. (Source: Supplementary Notes Source 1; Source 5)
138 ``
139

```

```

140 **Judge groundedness:** 5/5
141 **Judge answer relevancy:** 5/5
142
143 **Human groundedness (1-5):** 5
144 **Human answer relevancy (1-5):** 5
145
146 **Comments (free text):**
147
148 >
149
150 ---
151
152 ## Spot-check 5 – `conv_chattels_found` (conversational_followup)
153
154 **Question:** Does anything in our indexed materials suggest the answer changes if the chattel was found on the ground rather than handed over? Explain briefly.
155
156 **Agent answer (excerpt):**
157
158 ```
159 Yes – the materials suggest **the answer can change** if the chattel was **found on the ground** rather than **handed over**.
160
161 ## Brief explanation
162
163 For ordinary possession, the lecture says you need **physical control** plus **an intention to exclusively control** (Source: Lecture 3 Slide 4). The readings also say a person does **not** possess a chattel if they are **unaware of its existence**; possession only arises once they **know of it and assent to possessing it** (Source: Readings Week 3).
164
165 But the readings also show a **special rule for chattels found on premises**. In that setting, the occupier's rights may depend on whether they had, **before the item was found**, **manifested an intention to exercise control over the building and the things which may be upon it or in it** (Source: Readings Week 3). The materials say this is especially strong in a **private residence**, where that intention may be **presumed** and the householder will "almost invariably" possess the lost article, even without knowing it was there (Source: Readings Week 3).
166
167 ## So the practical difference is
168
169 - **Handed over / in someone's custody:** the normal knowledge-and-assent idea matters more.
170 - *
171
172 ...[truncated for readability]
173 ```
174
175 **Judge groundedness:** 5/5
176 **Judge answer relevancy:** 5/5
177
178 **Human groundedness (1-5):** 5
179 **Human answer relevancy (1-5):** 5
180
181 **Comments (free text):**
182
183 >
184
185 ---
186
187
188 ## Disagreement Summary (auto-computed)
189
190 _Computed from human ratings filled in above (n=5). Mean absolute error_
191 _ (MAE) between human score and judge score; lower is better. |Δ|≥2 counts_
192 _substantive disagreements per the convention at the top of this file._
193
194 | Metric | n | MAE | Exact agreement | Substantive disagreement (|Δ|≥2) |
195 |-----|---|-----|-----|-----|
196 | Groundedness | 5 | 0.00 | 5/5 | 0/5 |
197 | Answer relevancy | 5 | 0.00 | 5/5 | 0/5 |
198
199 ### Per-case disagreement
200
201 | Case | Family | Human/Judge G | |Δ| G | Human/Judge R | |Δ| R |
202 |-----|-----|-----|-----|-----|-----|
203 | `fact_perry_possessory` | factual_retrieval | 5/5 | 0 | 5/5 | 0 |
204 | `xmodal_moukataff_slide` | cross_modal_retrieval | 5/5 | 0 | 5/5 | 0 |
205 | `fact_fee_simple` | factual_retrieval | 5/5 | 0 | 5/5 | 0 |
206 | `conv_ap_followup` | conversational_followup | 5/5 | 0 | 5/5 | 0 |
207 | `conv_chattels_found` | conversational_followup | 5/5 | 0 | 5/5 | 0 |

```